

Projeto Final em Engenharia Informática

Musical Theory Trainer

COM GERAÇÃO PROCEDIMENTAL E AVALIAÇÃO AUTOMÁTICA

RELATÓRIO FINAL

João Paulo Ramos Ferreira, aluno 1800238

Orientador: Professor Pedro Duarte Pestana

21 de junho de 2026

Índice

Capítulo 1 - Introdução	6
1.1. DESCRIÇÃO DO PROJETO	6
1.2. ÂMBITO E ENQUADRAMENTO	6
1.3. NECESSIDADES IDENTIFICADAS.....	7
1.3.1. Musictheory.net	7
1.3.2. Teoria.com.....	7
1.3.3. Diferenciação e Proposta de Valor do <i>Musical Theory Trainer</i> (MTT)....	8
1.4. OBJETIVOS.....	8
1.5. O QUE SE PRETENDE FAZER.....	9
1.6. RESTRICÇÕES	9
1.7. RESULTADOS ESPERADOS.....	9
1.8. CALENDÁRIO GERAL	10
1.9. ESTRUTURA DO RELATÓRIO	10
Capítulo 2 - Desenho	11
2.1. STACK TECNOLÓGICA.....	11
2.1.1. Tecnologias de <i>frontend</i> (interface)	12
2.1.2. Tecnologias de <i>backend</i> (Motor e Lógica)	12
2.1.3. Persistência de Dados e Segurança	13
2.2. ARQUITETURA DO SISTEMA	13
2.3. ESTRUTURA DE DADOS	16
2.4. ALGORITMOS PARA A GERAÇÃO PROCEDIMENTAL	17
2.4.1. Fundamentação teórica da Geração Procedimental.....	18
2.5. JUSTIFICAÇÃO DE OPÇÕES E ALTERNATIVAS	18
2.6. ANÁLISE DE ESCABILIDADE E DESEMPENHO DA ARQUITECTURA	20
2.6.1. Escalabilidade Horizontal da Camada de Aplicação	20
2.6.2. Limitações de Escalabilidade Vertical na Persistência	20
2.7. INTERFACE DO UTILIZADOR	21
Capítulo 3 - Implementação	26
3.1. ESTRUTURA DE DECOMPOSIÇÃO DO TRABALHO (WBS).....	26
3.2. PRECEDÊNCIAS E DISTRIBUIÇÃO DE TAREFAS.....	27
3.3. CALENDARIZAÇÃO E PROGRESSO	27

3.4.	RELATÓRIO DE ESTADO DA IMPLEMENTAÇÃO.....	28
Capítulo 4 - Testes		29
4.1.	TESTES UNITÁRIOS AUTOMATIZADOS (MOTOR LÓGICO)	29
4.2.	TESTES FUNCIONAIS E DE INTERFACE.....	30
4.2.1.	CTo1: Registo de Novo Utilizador com Sucesso.....	31
4.2.2.	CTo2: Validação de Registo Duplicado	32
4.2.3.	CTo3: Validação de Login Inválido.....	32
4.2.4.	CTo4: Geração de Exercício	33
4.2.5.	CTo5 (Resposta Correta) e CTo6 (Resposta Errada)	33
4.2.6.	CTo7: Funcionalidade Áudio e CTo8: Persistência.....	34
4.2.7.	CTo9: Sugestões de estudo	35
Capítulo 5 - Conclusões		35
5.1.	Reflexão sobre os Resultados	36
5.2.	Dificuldades e Limitações	36
5.3.	Melhorias e Trabalho Futuro	38
Bibliografia		39
Anexo I – Código desenvolvido		40
	<i>frontend</i> - index.html	40
	<i>frontend</i> - app.js.....	42
	<i>frontend</i> - styles.css	50
	<i>backend</i> – main.py.....	52
	<i>backend</i> – models.py	55
	<i>backend</i> – music_logic.py.....	56
	<i>backend</i> – test_music_logic.py.....	60

Lista de Figuras

Figura 1 - Diagrama C4 Nível 1: Contexto do Sistema.....	14
Figura 2 - Diagrama C4 Nível 2: Contentores do Sistema	15
Figura 3 - Diagrama de Sequência: Ciclo de Geração de Exercício	15
Figura 4 - Estrutura de ficheiros do projeto	16
Figura 5 - Modelo de Dados: Diagrama de Classes UML	16
Figura 6 - Ecrã Inicial.....	22
Figura 7 - Ecrã para autenticação.....	22
Figura 8 - Ecrã após autenticação de utilizador registado e de utilizador sem registo..	22
Figura 9 - Exemplo de exercício para classificação de intervalos.....	23
Figura 10 - Exemplo de exercício para classificação de intervalos - resposta errada / correta.....	23
Figura 11 - Exemplo de exercício para identificação de escalas	24
Figura 12 - Exemplo de exercício para identificação de escalas - resposta errada / correta	24
Figura 13 - Exemplo de exercício para identificação de tonalidades.....	25
Figura 14 - Exemplo de exercício para identificação de tonalidades – resposta errada / correta.....	25
Figura 15 - Apresentação sem exibição da pauta e das métricas.....	26
Figura 16 - Resultado dos testes unitários automatizados.....	30
Figura 17 - Registo preenchido – Novo utilizador / métricas a zeros.....	31
Figura 18 - Tentativa de registo com nome de utilizador já existente.....	32
Figura 19 - Tentativa de autenticação com password incorreta	32
Figura 20 - Pauta desenhada corretamente. Quatro opções de resposta.....	33
Figura 21 -Resposta correta e incorreta, com apresentação da justificação pedagógica.	34
Figura 22 - Gráficos de recuperação de sessão histórica (utilizador JP2)	34
Figura 23 - Sugestão de estudo com base no tipo de exercício com menor taxa de acerto	35

Lista de Tabelas

Tabela 1 - Cronograma geral do projeto	10
Tabela 2 - Decisões de arquitetura e alternativas consideradas.....	18
Tabela 3 - Calendarização e estado das tarefas.....	27
Tabela 4 – Cenários para testes de funcionais e de interface	30

Resumo

O *Musical Theory Trainer* (MTT) é uma plataforma web interativa para apoio ao estudo autónomo da teoria musical, desenvolvida no âmbito da U.C. *Projeto de Engenharia Informática* da Licenciatura em Engenharia Informática da Universidade Aberta. O sistema implementa geração procedimental de exercícios através de um motor que calcula notas e suas propriedades em tempo real. A renderização usa SVG vetorial (*VexFlow*) e síntese *Web Audio* (*Tone.js*), sem recurso a ficheiros de áudio estáticos. A avaliação das respostas é automática, com visualização imediata da resposta e da respetiva justificação pedagógica. Permite o registo persistente do histórico de desempenho por utilizador registado ou a utilização por utilizadores sem registo.

A arquitetura assenta numa *Single Page Application* (SPA) em *JavaScript Vanilla*, que comunica via *API REST* com o *backend* em *Python* (*FastAPI*). A notação musical é gerada em formato vetorial (SVG) pela biblioteca *VexFlow*, a síntese de áudio é assegurada pelo *Tone.js*, a visualização analítica de métricas é suportada pelo *Chart.js*, e a persistência de dados é garantida por uma base de dados *SQLite* gerida através do *ORM SQLAlchemy*. O motor de geração procedimental implementa um algoritmo de ortografia diatónica e distratores pedagógicos inteligentes assegurando a correção teórica e ortográfica das pautas geradas e conferindo valor diagnóstico às opções de resposta geradas.

O Produto Mínimo Viável (MVP) encontra-se integralmente implementado e alojado em ambiente de produção na nuvem (*Render*¹) acedível através do endereço <https://pei-mta.onrender.com/>. Para além dos requisitos inicialmente previstos, a versão final inclui a gestão de perfis com autenticação, funcionalidades de treino auditivo avançado e um *dashboard* que orienta ativamente o estudo do utilizador com base nas áreas em que demonstrou maior dificuldade. A estabilidade da arquitetura e das invariantes musicais foi certificada com sucesso através de baterias de testes unitários automatizados.

Reproduz-se no Anexo I do presente relatório o código do projeto, que também está disponível no GitHub: <https://github.com/jopaferreira/PEI-MTA>.

¹ <https://render.com/>

Capítulo 1 - Introdução

Este capítulo apresenta o projeto *Musical Theory Trainer*. Contém a descrição do problema que pretende resolver e os objetivos fixados para o Produto Mínimo Viável (MVP). Descreve as restrições tecnológicas e apresenta a calendarização das tarefas.

1.1. DESCRIÇÃO DO PROJETO

O estudo da teoria musical requer prática regular e beneficia significativamente da avaliação para a consolidação de conceitos. O *Musical Theory Trainer* – MTT pretende ser uma plataforma web interativa baseada em geração procedimental, desenhada para apoiar o estudo autónomo de estudantes de música e autodidatas. Ao contrário dos métodos tradicionais em papel - limitados pela ausência de reprodução sonora, esta solução combina a visualização em notação musical (pauta) com áudio sintetizado. São gerados exercícios para identificação de intervalos, escalas e tonalidades, devolvendo a avaliação automática das respostas e esclarecimentos teóricos sempre que a resposta indicada não seja a correta.

O ensino da música assistido por computador está consolidado na pedagogia musical moderna, revelando-se um aliado indispensável no desenvolvimento do ouvido interno e da literacia tonal (*Desain & Honing*, 1992). O treino de competências como a identificação de intervalos, o reconhecimento de modos e a leitura de armações de clave exige prática sistemática e, idealmente, individualizada. A literatura indica que a eficácia do treino autónomo fora da sala de aula está ligada à natureza da resposta fornecida pelo sistema. De acordo com *Hattie e Timperley* (2007), a resposta formativa imediata (quando o estudante submete uma resposta) possui o maior índice de impacto na retenção de conceitos. No ensino tradicional ou em plataformas sem correção automática explicativa um erro cometido pelo aluno costuma persistir até à próxima sessão presencial com o docente. Ao juntar a visualização em tempo real com a síntese de áudio e com uma componente de diagnóstico pedagógica, o ambiente promove a autonomia autorregulada, transformando o erro em fonte de conhecimento imediato.

1.2. ÂMBITO E ENQUADRAMENTO

O projeto enquadra-se no domínio do desenvolvimento de ferramentas de e-learning para a aprendizagem musical. Trata-se de uma *Single Page Application* (SPA) com o processamento visual e sonoro realizado pelo cliente para garantir uma interface fluida visando proporcionar uma experiência de utilização agradável. A arquitetura do sistema

assenta num *Frontend* em *JavaScript Vanilla* e num *backend* em *Python (FastAPI)*, apoiado por uma base de dados *SQLite* para o armazenamento de métricas.

1.3. NECESSIDADES IDENTIFICADAS

O desenvolvimento de sistemas de Ensino de Música Assistido por Computador (*Computer-Assisted Music Instruction - CAMI*) deve ter em consideração o rigor científico da teoria musical e a eficiência da plataforma tecnológica. Para enquadrar o *Musical Theory Trainer (MTT)* no panorama atual efetuou-se uma análise comparativa sucinta com as duas principais plataformas de referência globais: o *Musictheory.net* e o *Teoria.com*. Esta avaliação teve por base três critérios de Engenharia: o modelo de persistência, o método de geração de conteúdos e a infraestrutura de processamento áudio.

1.3.1. Musictheory.net

O *Musictheory.net* é uma das ferramentas mais difundidas no ensino musical, destacando-se por apresentar uma interface gráfica muito responsiva. Sob a perspetiva de arquitetura de dados, a plataforma apresenta limitações na sua versão gratuita uma vez que não oferece um sistema de persistência de resultados para os utilizadores. O progresso do utilizador é volátil, retido apenas temporariamente através de cookies do *browser*, sendo necessária a subscrição de aplicações móveis pagas para manter o histórico ao longo do tempo. O motor de exercícios é de geração estocástica simples sem módulos explicativos quando o utilizador comete um erro.

1.3.2. Teoria.com

O *Teoria.com* oferece uma cobertura alargada e rigorosa de conceitos avançados sendo uma referência para estudantes de conservatório. A experiência proporcionada ao utilizador mostra-se algo obsoleta uma vez que a interface é pouco responsiva. Com esta plataforma as curvas de aprendizagem são íngremes para utilizadores principiantes. A reprodução auditiva recorre a ficheiros áudio estáticos pré-gravados ou ao mapeamento de sequências MIDI rígidas. Esta abordagem aumenta o peso arquitetural da plataforma e retira flexibilidade para a introdução de variações tímbricas em tempo real.

1.3.3. Diferenciação e Proposta de Valor do *Musical Theory Trainer* (MTT)

O sistema desenvolvido neste projeto tenta colmatar as lacunas das plataformas referidas ao introduzir uma arquitetura assente em três eixos de inovação para o ecossistema de código aberto:

- **Geração Procedimental Pura vs. Repositórios Estáticos:** Enquanto os sistemas tradicionais dependem de questionários fixos ou do armazenamento de imagens de pautas, o MTT calcula as matrizes de meios-tons e armações de clave em tempo real no *backend* (Python/FastAPI). Esta abordagem permite variabilidade sem necessidade de armazenamento em disco.
- **Síntese Web Audio:** Através da integração da *framework Tone.js*, o MTT efetua a síntese de frequências em tempo real diretamente no hardware do cliente o que liberta o servidor dos fluxos de transmissão de áudio (não há *streaming*). Também permite ao utilizador escolher o timbre do instrumento melhorando o estímulo auditivo.
- **Análise de Desempenho Gratuita:** Ao contrário do *Musictheory.net*, o MTT tem persistência (*SQLite* + *SQLAlchemy ORM*) que devolve ao utilizador um painel analítico (*Chart.js*) sem custos. Os dados cronológicos são armazenados sendo calculada a potencial lacuna de conhecimento mais expressiva do utilizador pelo que o histórico assume relevância como indicador para o foco de estudo.

1.4. OBJETIVOS

Para satisfazer as necessidades identificadas, foram definidos os seguintes objetivos:

- **Geração procedimental:** desenvolver um motor capaz de calcular notas e escalas em tempo real, sem recurso a exercícios estáticos que poderiam ser repetitivos.
- **Notação e áudio:** desenhar pautas musicais com rigor e reproduzir o som correspondente sem recurso a ficheiros áudio pré-gravados.
- **Avaliação automática:** validar as respostas do utilizador, em formato escolha múltipla e fornecer justificações teóricas em caso de erro.
- **Monitorização de progresso:** registar o histórico de desempenho do utilizador, disponibilizando métricas de sucesso/insucesso.

1.5. O QUE SE PRETENDE FAZER

Pretende-se implementar o projeto com base numa arquitetura com três componentes principais:

- *backend*: implementação da lógica musical e das rotas de API com recurso ao *framework FastAPI*.
- *Frontend*: construção de uma SPA que integra as bibliotecas *VexFlow* para visualização de notação musical, *Tone.js* para síntese de áudio e *Chart.js* para a representação gráfica das métricas de desempenho no *dashboard*.
- Base de dados: gestão de utilizadores e tentativas através do *ORM SQLAlchemy*, com persistência em *SQLite*.

1.6. RESTRIÇÕES

O desenvolvimento teve em consideração as seguintes restrições técnicas e de segurança:

- **Políticas de áudio**: os browsers modernos bloqueiam a reprodução automática, exigindo uma interação explícita do utilizador para iniciar o contexto de áudio.
- **Ortografia musical**: o motor procedimental deve assegurar o uso correto de acidentes (sustenidos e bemóis).
- **Escalabilidade no MVP**: a adoção de *SQLite* circunscreve o sistema a acessos locais ou de baixa concorrência, não sendo adequada para ambientes de produção de maior escala.

1.7. RESULTADOS ESPERADOS

O resultado esperado é uma aplicação funcional que permita ao utilizador, num único ecrã, gerar uma nova questão, ouvir uma reprodução sonora do exercício, submeter a resposta, validar a resposta submetida com indicação pedagógica em caso de erro e obter a taxa de resposta corretas atualizada de forma instantânea. A aferição do sucesso do MVP traduz-se na implementação completa e na integração eficaz dos requisitos funcionais nucleares (*Must-have*), nomeadamente:

- a geração procedimental correta dos desafios musicais;
- a sua respetiva representação visual e reprodução sonora sem falhas;
- a avaliação automática das respostas submetidas;
- registo persistente do histórico de desempenho do utilizador.

O critério de desempenho para o objetivo estabelecido é um tempo de resposta da API inferior a um segundo por pedido, garantindo a fluidez do ciclo de interação do utilizador.

1.8. CALENDÁRIO GERAL

O cronograma detalhado do projeto é apresentado na tabela seguinte:

Tabela 1 - Cronograma geral do projeto

Semanas	Conteúdo planeado	Marco
Sem. 1-4	Proposta, Requisitos e Arquitetura	Proposta (25 mar)
Sem. 5-7	Wireframes e Implementação do Núcleo (API/SPA)	Demo interna
Sem. 8	Redação do Relatório Intermédio	Intercalar (6 mai)
Sem. 9-12	Implementação de Exercícios e Áudio	
Sem. 13-14	Testes funcionais e Polimento da Interface	
Sem. 15-16	Conclusões e Entrega Final	Final (24 jun)

1.9. ESTRUTURA DO RELATÓRIO

O presente relatório encontra-se organizado da seguinte forma:

- **Capítulo 1 – Introdução:** presente capítulo.
- **Capítulo 2 – Desenho:** explicita as tecnologias adotadas, apresenta os diagramas de arquitetura e as decisões de interface do utilizador.

- **Capítulo 3 – Implementação:** descreve a decomposição de tarefas e o estado atual do desenvolvimento.
- **Capítulo 4 – Testes:** detalha os procedimentos de teste realizados e respectivos resultados.
- **Capítulo 5 – Conclusões:** Apreciação global do trabalho desenvolvido.
- **Anexo I** – Código integral do projeto.

Capítulo 2 - Desenho

Este capítulo explicita a arquitetura do sistema *Musical Theory Trainer*, justificando as escolhas tecnológicas. Apresenta a modelação de dados, a lógica algorítmica e o desenho final da interface.

2.1. STACK TECNOLÓGICA

A *stack* tecnológica foi escolhida para garantir uma aplicação leve, rápida e de fácil manutenção:

- **Linguagens e Frameworks:** *Python 3.11* com a *framework FastAPI* para o desenvolvimento do servidor de API, garantindo um bom desempenho e a geração automática de documentação². No lado do cliente, optou-se por *HTML5*, *CSS3* e *JavaScript Vanilla*, para a implementação do modelo de *Single Page Application* (SPA).
- **Bibliotecas de Terceiros:**
 - *VexFlow*: Biblioteca para a representação da notação musical em formato SVG (*Scalable Vector Graphics*);
 - *Tone.js: Framework* para síntese de áudio;
 - *Chart.js*: Biblioteca para a geração dos gráficos de evolução temporal e desempenho no *dashboard*.
 - *SQLAlchemy*: ORM (*Object-Relational Mapper*) para a gestão da base de dados através de classes *Python* (Fowler, 2002);
 - *Pydantic*: Utilizada pelo *FastAPI* para a validação dos dados à entrada da API.

² <http://127.0.0.1:8000/docs> (servidor local, p.ex.);
<https://pei-mta.onrender.com/docs> (render)

- *Pytest*: Framework utilizada para a execução da bateria de testes unitários automatizados ao motor procedimental.

2.1.1. Tecnologias de *frontend* (interface)

Descrevem-se as tecnologias e conceitos que foram utilizados na construção do *frontend*, respetivas funcionalidades e motivações arquiteturais:

- *JavaScript Vanilla* - Linguagem de programação em estado puro, sem o uso de frameworks reativas (como *React* ou *Angular*). Responsável pela lógica da *interface*, manipulação direta do *DOM* e gestão de eventos (como os cliques nos botões). Visa garantir uma *Single Page Application* (SPA) leve, com carregamento rápido e livre de configurações de compilação complexas, o que se alinha perfeitamente com a agilidade exigida para um Produto Mínimo Viável (MVP).
- *VexFlow* - Biblioteca de código aberto baseada em *JavaScript* para a produção visual da notação musical em formato vetorial (SVG) no ecrã. Sendo considerado o standard na indústria para *renderização* musical web, permite desenhar pautas limpas e escaláveis no *browser* sem recorrer a imagens pré-geradas no servidor, viabilizando o requisito da geração procedimental de exercícios.
- *Tone.js* - *Framework* responsável pela síntese e sequenciação de áudio no browser. Elimina a necessidade de armazenar, gerir ou descarregar ficheiros de áudio estáticos (como MP3 ou WAV). Permite que o som seja sintetizado em tempo real, gerando os timbres das notas calculadas matematicamente pelo motor.
- *Chart.js* - Biblioteca gráfica de *JavaScript* para a visualização de dados. É facilmente integrável e é leve. Foi utilizada para transformar as métricas de desempenho (recebidas do *backend*) em gráficos de barras e linhas dinâmicos, essenciais para o funcionamento visual do *Dashboard* e do módulo de sugestões de estudo.

2.1.2. Tecnologias de *backend* (Motor e Lógica)

Descrevem-se as tecnologias e conceitos que foram utilizados na construção do *backend*, respetivas funcionalidades e motivações arquiteturais:

- *FastAPI* - *Framework* em *Python* destinada à construção do servidor de API³. Escolhida pela sua performance, simplicidade de código e pela geração

³ *Application Programming Interface* – conjunto de regras e definições que permite que diferentes blocos de código, bibliotecas, aplicações ou sistemas comuniquem entre si de forma padronizada.

automática da documentação interativa (*Swagger UI*), o que facilita os testes de comunicação de dados.

- *Pydantic* - Biblioteca em *Python* para validação de dados. Funciona conjuntamente com o *FastAPI* e garante que qualquer dado que entra no servidor (ex: as submissões de tentativas do utilizador) cumpre estritamente o formato esperado, bloqueando pedidos malformados antes de chegarem à lógica principal.
- *os* - Módulo nativo da linguagem *Python* que fornece funções para interagir com o sistema operativo da máquina onde corre o *backend*. Utilizado para construir caminhos de ficheiros absolutos, garantindo que o servidor *backend* encontra a pasta do *Frontend*, independentemente do ambiente onde o código está a correr (localmente ou num servidor na *cloud*).

2.1.3. Persistência de Dados e Segurança

Apresenta-se uma descrição estruturada, sucinta e rigorosa das tecnologias e conceitos que utilizados para implementar a persistência de dados e a segurança, com as respetivas funcionalidades e motivações arquiteturais:

- *SQLAlchemy* - Mapeador Objeto-Relacional (ORM) para *Python*, que traduz comandos de linguagem de programação em *queries SQL*. Permite consultas à base de dados, converte registos em objetos *Python* fáceis de manipular, previne vulnerabilidades como injeção de SQL e torna o código independente do motor de base de dados, permitindo migrar do *SQLite* para o *PostgreSQL* futuramente sem reescrever a lógica.
- *hashlib* - Biblioteca nativa de criptografia do *Python*. Implementada para aplicar o algoritmo *SHA-256* às credenciais dos utilizadores. Garante que as palavras-passe não são guardadas em texto limpo, impossibilitando a sua leitura direta mesmo que o ficheiro da base de dados seja comprometido.

2.2. ARQUITETURA DO SISTEMA

Utiliza-se uma arquitetura cliente-servidor desacoplada. O *Frontend* funciona como contentor *Web App* que comunica via pedidos HTTP (JSON) com o contentor *API* do *backend*.

- *frontend* (SPA): Gere o estado da interface, processa o desenho das pautas (*VexFlow*) e assegura a reprodução do som (*Tone.js*) localmente. A avaliação de respostas (certo/errado) é processada *client-side* por comparação com o *payload*

JSON recebido, libertando o servidor para garantir escalabilidade num contexto de avaliação formativa.

- *backend* (API REST): Responsável pelo motor de geração musical procedimental e pelas rotas de persistência. O servidor utiliza caminhos absolutos do sistema operativo (módulo *os*) para servir nativamente a SPA, assegurando estabilidade no alojamento em nuvem (*Render*).
- Base de Dados: Armazenamento local em *SQLite*, que dispensa configuração de infraestrutura. A segurança das credenciais é garantida pelo algoritmo de *hashing SHA-256* (módulo *hashlib*).
- *API REST* – arquitetura para sistemas distribuídos (*REST - Representational State Transfer*) que usa protocolos HTTP (GET, POST, DELETE) para comunicar e transferir de dados em formato JSON⁴. Permite um desacoplamento entre o *Frontend* e o *backend*, poupando memória RAM e permitindo escalabilidade para múltiplos utilizadores em simultâneo.
- *Render* - Plataforma de alojamento na nuvem. Permite realizar o *deploy* (colocação em produção) da aplicação diretamente a partir do repositório *GitHub* sem necessidade de configurar servidores virtuais. Acomoda um serviço web suportado por ficheiros locais, como o *SQLite*.

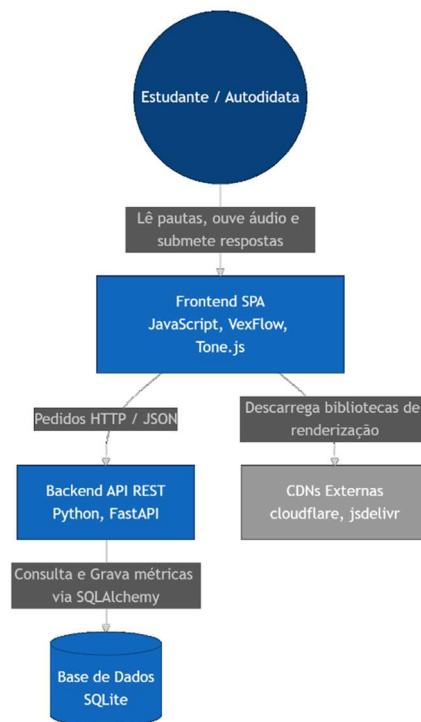


Figura 1 - Diagrama C4 Nível 1: Contexto do Sistema

⁴ *JavaScript Object Notation* - formato leve para troca de dados, utilizado para enviar e receber informações entre sistemas.

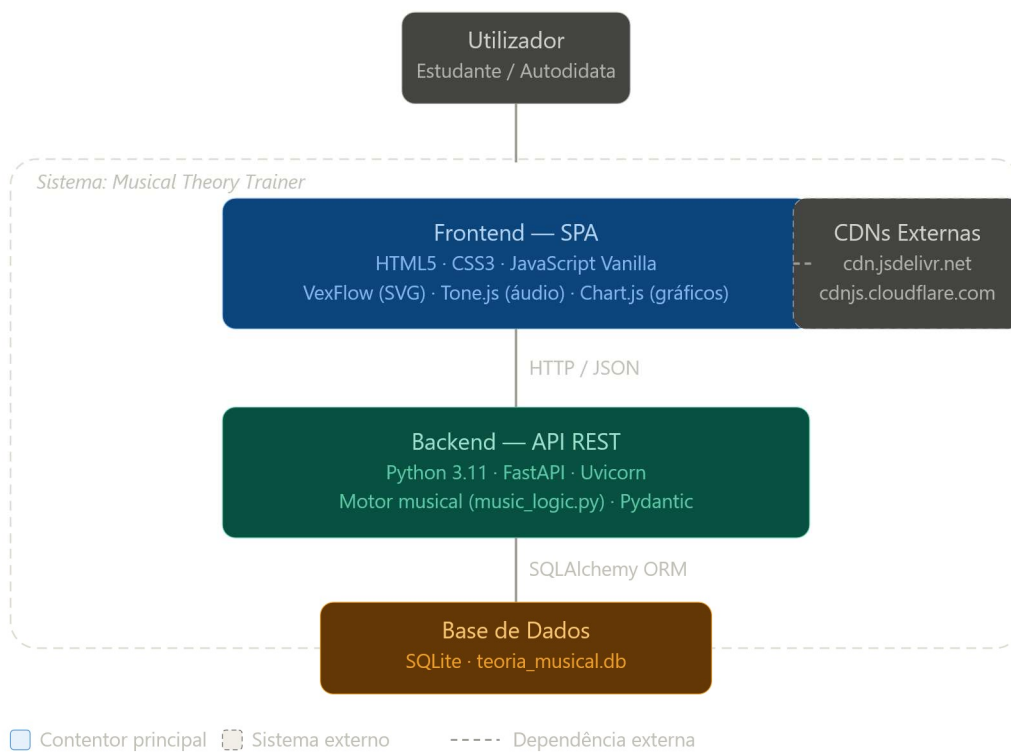


Figura 2 - Diagrama C4 Nível 2: Contentores do Sistema

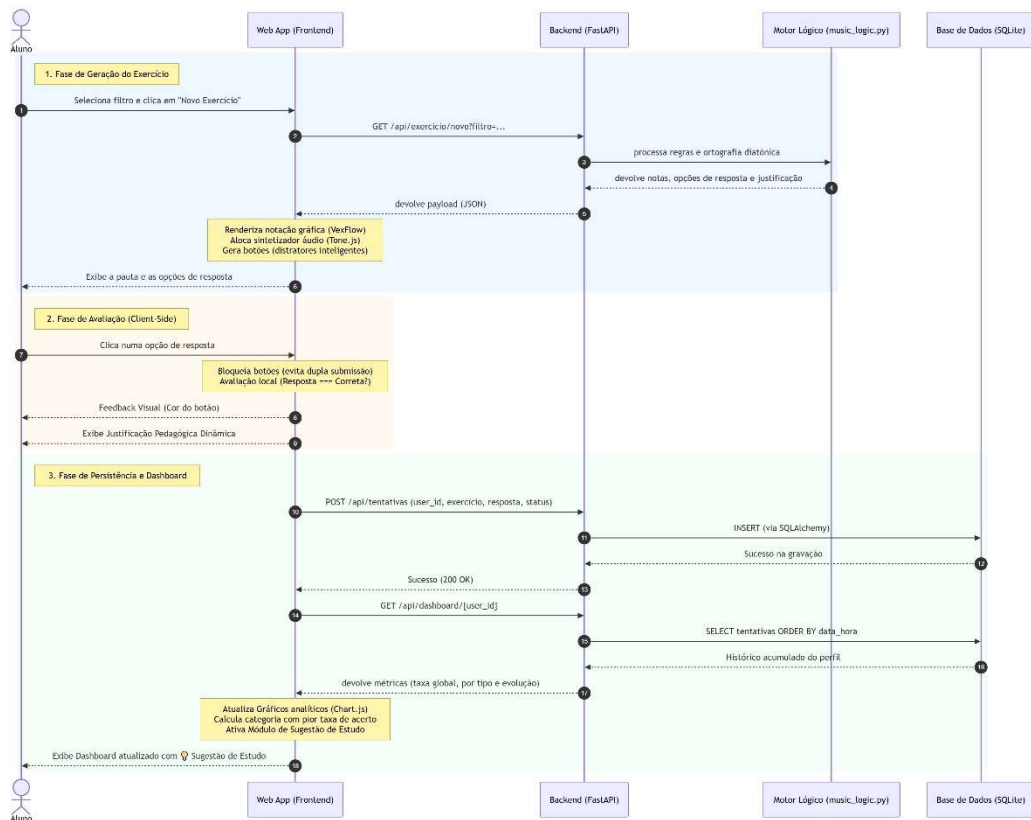


Figura 3 - Diagrama de Sequência: Ciclo de Geração de Exercício

PEI-MTT/	<-- Raiz do Projeto
├── backend/	<-- Contentor da API REST e Motor Lógico
├── main.py	<-- Servidor FastAPI, Rotas e Lógica de Autenticação
├── models.py	<-- Definição do Esquema Relacional (SQLAlchemy)
├── music_logic.py	<-- Motor de Geração Procedimental Ortográfica
├── test_music_logic.py	<-- Bateria de Testes Unitários Automatizados (pytest)
└── teoria_musical.db	<-- Base de Dados Relacional Embebida (SQLite)
├── frontend/	<-- Contentor da Web App (SPA)
├── index.html	<-- Estrutura DOM e Ancoragem de CDNs
├── app.js	<-- Controlor de Interface, Gráficos, Áudio e Clientes
└── styles.css	<-- Estilização Visual e Animações do Splash Screen

Figura 4 – Estrutura de ficheiros do projeto

2.3. ESTRUTURA DE DADOS

A base de dados suporta a gestão de tentativas e métricas com autenticação individual:

- **Tabela utilizadores:** Armazena o perfil do utilizador (*id*, *username*, *password_hash*, *data_registo*). Suporta registos múltiplos com isolamento de dados.
- **Tabela tentativas:** Regista cada exercício resolvido (*id*, *utilizador_id*, *tipo_exercicio*, *detalhe*, *resposta_dada*, *correta*, *data_hora*). O campo *data_hora* utiliza *timezone.utc* para garantir consistência temporal global, ou seja, é independente da localização física do servidor e das eventuais mudanças entre hora de verão e inverno.
- **Relacionamento:** Existe uma relação de "um para muitos" entre utilizadores e tentativas, permitindo calcular o histórico do progresso individual para cada utilizador.

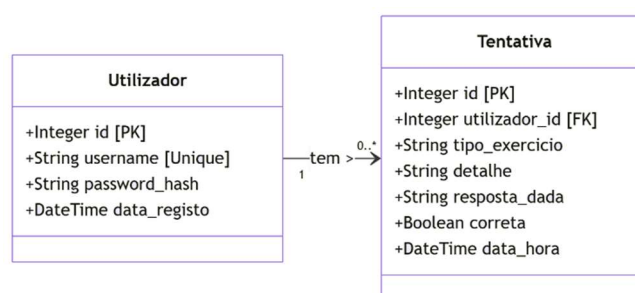


Figura 5 - Modelo de Dados: Diagrama de Classes UML

2.4. ALGORITMOS PARA A GERAÇÃO PROCEDIMENTAL

A geração procedimental é um paradigma de engenharia de software em que os dados ou conteúdos são criados com base em algoritmos em tempo real por oposição à sua criação manual prévia e posterior armazenamento estático em bases de dados. No *Musical Theory Trainer*, a adoção deste conceito significa que o sistema não recorre a um repositório exaustivo de exercícios predefinidos, imagens de pautas pré-desenhadas ou ficheiros de áudio pré-gravados. Ao invés, a cada novo pedido do utilizador, o motor alocado no *backend* calcula e constrói instantaneamente as propriedades musicais (notas, intervalos, modos ou armações de clave) com base num conjunto estrito de regras teóricas. Esta abordagem de arquitetura traz vantagens estruturais e pedagógicas significativas ao Produto Mínimo Viável (MVP):

- **Escalabilidade e Leveza:** Elimina a necessidade de armazenar ficheiros multimédia (SVG e MP3/WAV), reduzindo drasticamente o peso e os custos de infraestrutura do sistema.
- **Variabilidade:** Garante que o utilizador raramente se depara com a mesma exata permutação visual e auditiva de forma repetida, desincentivando a memorização e promovendo o verdadeiro raciocínio musical.
- **Tutoria:** Permite que os botões de opções erradas (distratores) e a caixa de justificação pedagógica final sejam gerados em tempo real, utilizando no seu texto as notas exatas acabadas de sortear pelo algoritmo.

O núcleo da aplicação assenta num motor de geração procedimental delimitado e rigoroso:

- **Ortografia Diatónica⁵ e Espaço Delimitado:** Para evitar a geração de enarmonias absurdas (por exemplo: apresentar um Lá Sustenido em vez de um Si Bemol, quando teoricamente incorreto, ou utilização de duplos sustenidos, difíceis de compreender para iniciantes), o espaço musical base foi restrito a 5 notas fundamentais. Um Dicionário de Mapeamento atribui o som (meios-tons) à letra ortograficamente correta do alfabeto musical.
- **Distratores Inteligentes:** Não se utilizam opções de resposta aleatórias. O algoritmo produz distratores que mapeiam erros prováveis dos utilizadores, conferindo valor diagnóstico ao erro. Nas tonalidades, são forçadas opções relativas (eixo maior/menor da mesma armação) ou vizinhas de contagem. Nos modos e intervalos, as opções são agrupadas por semelhança teórica.

⁵ Princípio da teoria musical que estabelece que cada nota de uma escala ou tonalidade deve ser representada por um nome de nota distinto, respeitando a sequência das sete letras musicais (Kostka, Payne & Almén, 2018).

- **Justificações Dinâmicas Contextualizadas:** Quando o utilizador erra, o motor produz uma explicação que usa as notas exatas presentes no ecrã. Demonstra a localização exata dos meios-tons nas escalas e contabiliza os graus e saltos nos intervalos apresentados.

2.4.1. Fundamentação teórica da Geração Procedimental

A Geração Procedimental traduz-se na criação de dados com base em algoritmos, em tempo real, alicerçados em regras matemáticas e lógicas estruturadas, (Togelius et al., 2011). Embora historicamente associado ao design de ambientes e dinâmicas em videojogos, o paradigma da Geração Procedimental tem sido progressivamente adotado na Engenharia de Software Educativa para mitigar o chamado "gargalo de conteúdos" (*content bottleneck*).

Nos Sistemas com Tutoriais Inteligentes, a transição de repositórios estáticos (bases de dados preenchidas manualmente com perguntas fixas) para motores procedimentais dinâmicos altera profundamente a experiência de aprendizagem. Conforme defendido por Shute (2008), a variabilidade gerada por vias procedimentais impede a memorização mecânica das respostas por parte do estudante, forçando o desenvolvimento do verdadeiro raciocínio abstrato e analítico sobre a matéria - neste caso, a decodificação de estruturas da teoria musical. Adicionalmente, a leveza da arquitetura que resulta da ausência de ficheiros multimédia pesados (.svg ou .mp3) valida a Geração Procedimental como uma abordagem preferencial para o desenvolvimento de aplicações e escaláveis.

2.5.JUSTIFICAÇÃO DE OPÇÕES E ALTERNATIVAS

As decisões foram tomadas com base nas exigências de um MVP, conforme se explicita na tabela seguinte:

Tabela 2 - Decisões de arquitetura e alternativas consideradas

Opção Tomada	Justificação	Alternativa Rejeitada
<i>Python/FastAPI</i>	Sintaxe robusta para algoritmos matemáticos e validação nativa de dados via <i>Pydantic</i> . Geração automática	Node.js / Express: adequado para I/O assíncrono, mas com menor expressividade para algoritmos de geração procedimental.

Opção Tomada	Justificação	Alternativa Rejeitada
	de documentação interativa (<i>Swagger UI</i> ⁶).	
<i>SQLite</i> + <i>SQLAlchemy</i>	Base de dados embebida que dispensa instalação de infraestrutura adicional, facilitando a distribuição e execução do projeto em qualquer máquina. O uso do ORM permite migrar para PostgreSQL com alteração mínima de código no futuro.	PostgreSQL/MySQL: Desnecessário para a fase atual de provas de conceito sem concorrência elevada.
SPA com <i>VexFlow</i> e <i>Tone.js</i>	O processamento visual e do áudio é executado no cliente, libertando o servidor. O <i>VexFlow</i> gera notação vetorial (SVG) precisa e escalável. O <i>Tone.js</i> sintetiza áudio em tempo real, eliminando a necessidade de ficheiros pré-gravados.	Aplicação nativa (<i>iOS/Android</i>): fora do âmbito do MVP e limitaria o acesso universal via browser. Imagens e áudio estáticos (PNG/MP3): incompatíveis com o requisito de geração procedimental.
API Stateless e Validação Client-Side	Permite uma resposta instantânea e poupa RAM no servidor, sendo adequada para ferramentas de Avaliação Formativa.	Validação Server-Side: Exigiria a persistência temporária do exercício no servidor (Modo de Exame). Transitado para trabalho futuro.
Geração Procedimental Delimitada	Circunscreve a geração a 5 notas base para garantir a precisão ortográfica e legibilidade visual na pauta para utilizadores iniciantes.	Espectro Aleatório Livre: Geraria casos teóricos ilegíveis ou acidentes duplos excessivamente complexos.
Segurança Básica (SHA-256 sem <i>salt</i>) e LocalStorage	Permite isolar os perfis dos utilizadores de forma simples, sendo suficiente para o MVP. Armazenamento local é utilizado exclusivamente para persistência do ID de utilizador, sem persistência de dados sensíveis	Sessões JWT e Bcrypt (Salts): Opções robustas para ambientes comerciais, mas que acrescentariam complexidade de infraestrutura e <i>middlewares</i> .

⁶ <http://127.0.0.1:8000/docs> (servidor local, p.ex.);
<https://pei-mta.onrender.com/docs> (render)

2.6. ANÁLISE DE ESCALABILIDADE E DESEMPENHO DA ARQUITECTURA

O desenho da arquitetura do *Musical Theory Trainer* foi efetuado tendo em vista a escalabilidade, dividindo-se em duas dimensões da engenharia de sistemas: a escalabilidade da camada de computação (*backend*) e a escalabilidade da camada de armazenamento (base de dados).

2.6.1. Escalabilidade Horizontal da Camada de Aplicação

A arquitetura cliente-servidor desacoplada baseada em funções assíncronas (*FastAPI*) e numa *API REST* confere ao sistema uma elevada capacidade de escalabilidade na camada de computação. Esta eficiência assenta em três pilares estruturais:

- **Natureza *Stateless* da API:** O servidor não armazena sessões ou estados de navegação em memória RAM, tratando cada pedido HTTP de forma isolada. Isto permite clonar e distribuir o contentor do *backend* por múltiplos servidores em paralelo sem risco de dessincronização de dados.
- **Processamento delegado:** Toda a computação da interface - incluindo a renderização gráfica das pautas em SVG (*VexFlow*), a síntese das frequências áudio (*Tone.js*), o processamento dos gráficos (*Chart.js*) e a própria validação da resposta - ocorre localmente no *hardware* do utilizador, libertando o CPU do servidor destas cargas, o *backend* assume uma função quase exclusiva de encaminhamento de dados leves em formato JSON, maximizando o número de utilizadores simultâneos suportados por núcleo de processador.

2.6.2. Limitações de Escalabilidade Vertical na Persistência

A camada de persistência (*SQLite*) representa o principal elemento de afunilamento (*bottleneck*) vertical do Produto Mínimo Viável:

- **Bloqueio ao nível da base de dados:** O *SQLite* opera sobre um único ficheiro em disco. Sempre que ocorre uma operação de escrita (como a submissão de um registo ou tentativa de exercício via *ORM SQLAlchemy*), o ficheiro é bloqueado de forma exclusiva. Num cenário de concorrência elevada com dezenas de escritas em simultâneo, os utilizadores podem observar latência ou falhas de *timeout*.
- **Vulnerabilidade dos identificadores de cliente:** O mecanismo atual depende da transmissão do identificador numérico de utilizador (*utilizador_id*) e da persistência em *localStorage* no cliente. À medida que o volume de dados aumenta, a ausência de encriptação impede a segregação segura e escalável do

tráfego. O modelo atual satisfaz perfeitamente os requisitos de latência inferior a um segundo estipulados para o MVP em ambiente académico, mas a transição para um ambiente de produção real de larga escala exigirá o desacoplamento do armazenamento através da migração para um SGBD relacional distribuído (*PostgreSQL*), conforme detalhado no plano de trabalho futuro.

2.7. INTERFACE DO UTILIZADOR

A *interface* foi desenhada e aperfeiçoada para ser intuitiva e simples, proporcionando ferramentas de controlo e monitorização do estudo.

- *Splash Screen*: Ecrã de carregamento animado com um temporizador de 2 segundos para garantir que os recursos (*VexFlow/Tone.js/Chart.js*) estão prontos.
- Ecrã Principal: Contentor que apresenta a pauta musical (SVG) e as opções de resposta em formato de escolha múltipla, recorrendo a botões.
- Visualização: Utilização de cores com significado para validação imediata (verde para correto, vermelho para incorreto).
- Menu de Controlo (Filtros e Timbres): O utilizador pode escolher treinar apenas um tipo de exercício específico através de *dropdowns*. Também pode escolher o timbre do som a reproduzir, alterando o sintetizador de som (Som Genérico, Sintetizador Suave, Jogo Retro (8-bit), Sintetizador Metálico, Piano Elétrico, Sopro, Guitarra / Corda), enriquecendo o estímulo auditivo.
- Treino Auditivo: Uma funcionalidade (*checkbox*) que oculta a pauta (*VexFlow*) permitindo ao utilizador resolver o exercício baseando-se apenas na audição através da identificação das relações tocadas pelo *Tone.js*.
- *Dashboard* Analítico: O painel inferior recorre à biblioteca *Chart.js* para desenhar um gráfico de barras (taxa de sucesso por categoria musical) e um gráfico de linhas (evolução temporal entre sessões). O sistema processa estes dados para alimentar um "módulo de sugestões de estudo", que aponta automaticamente o tópico com pior taxa de acerto, sugerindo-o ativamente como foco da próxima sessão de treino.

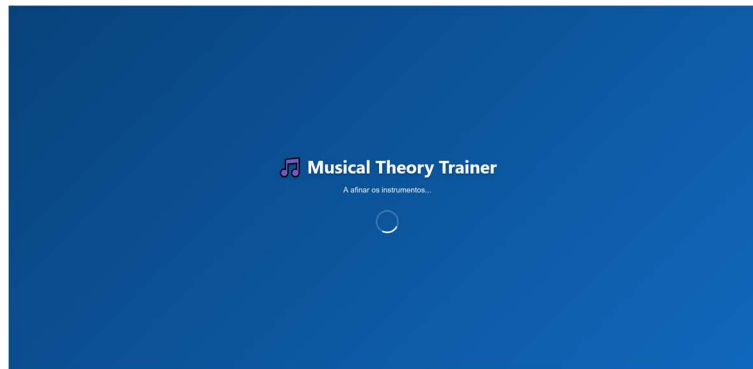


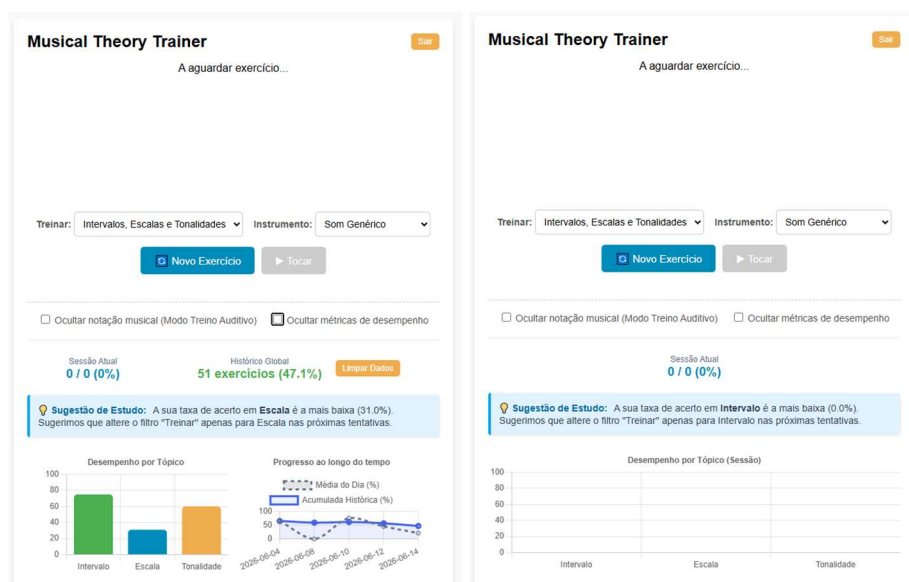
Figura 6 - Ecrã Inicial

Autenticação

Inicie sessão para guardar os seus resultados.

Entrar
Criar Conta
Utilizar sem registo

Figura 7 – Ecrã para autenticação



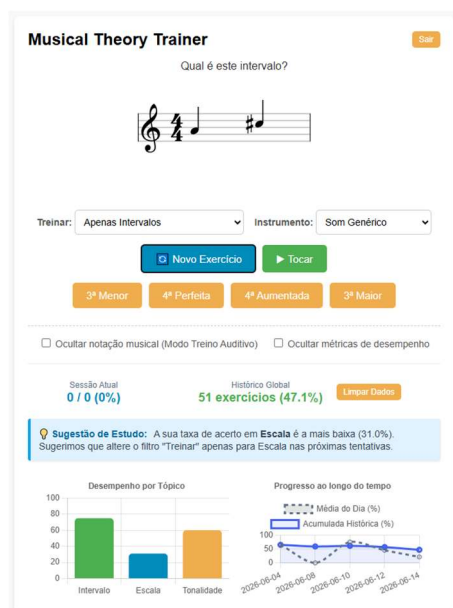


Figura 9 - Exemplo de exercício para classificação de intervalos

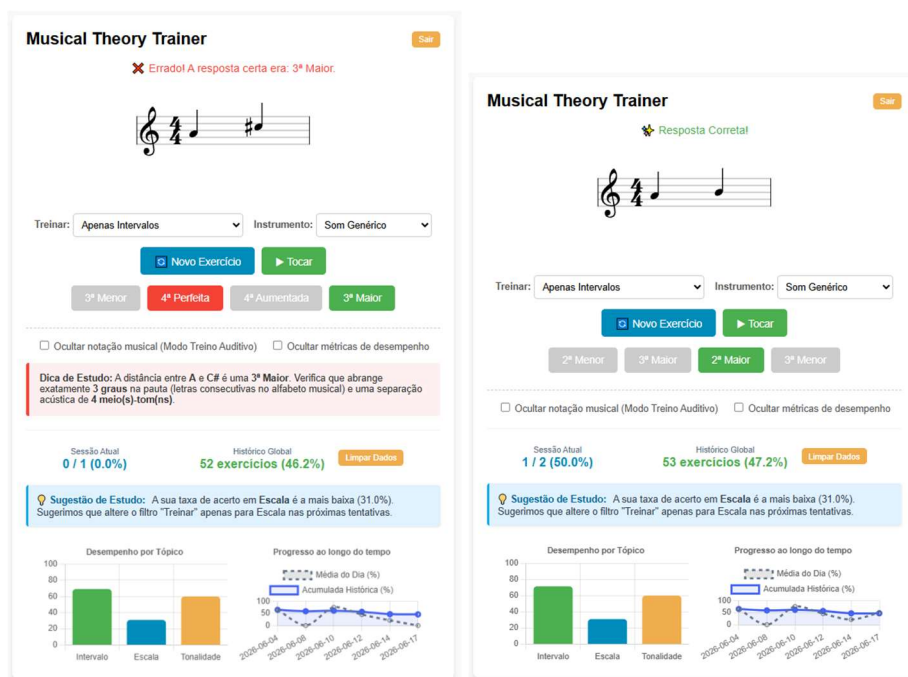


Figura 10 - Exemplo de exercício para classificação de intervalos - resposta errada / correta

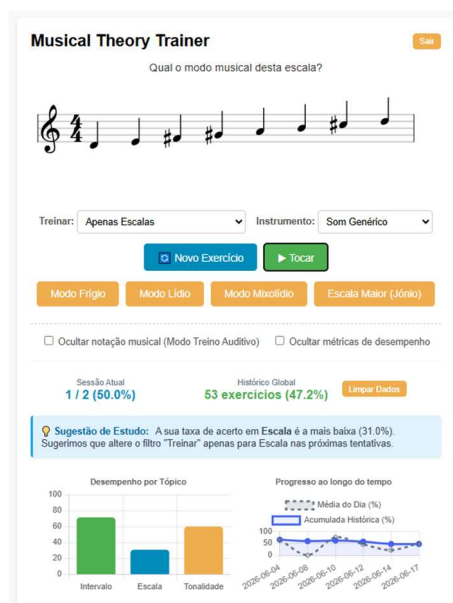


Figura 11 - Exemplo de exercício para identificação de escalas

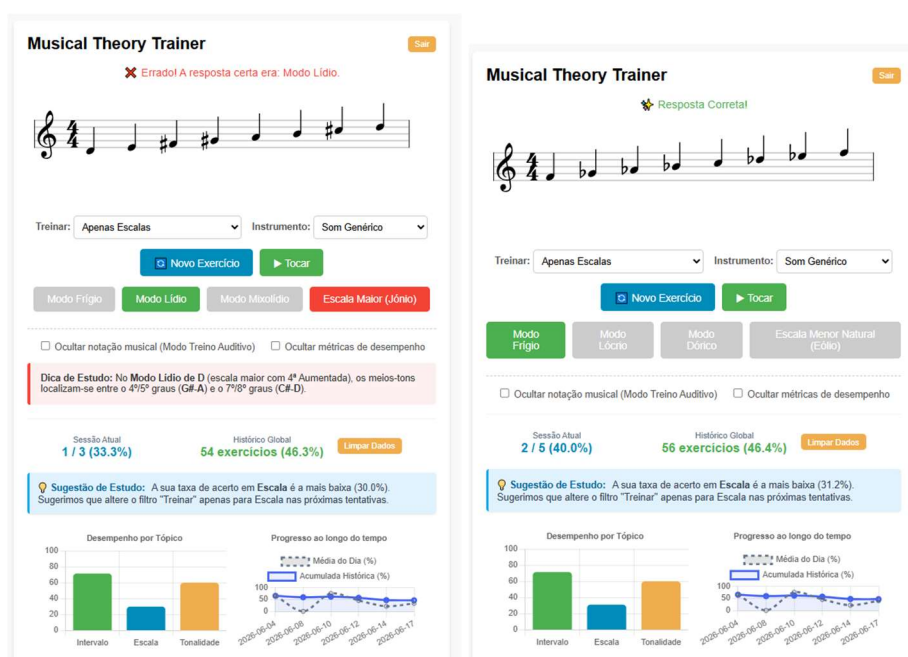


Figura 12 - Exemplo de exercício para identificação de escalas - resposta errada / correta

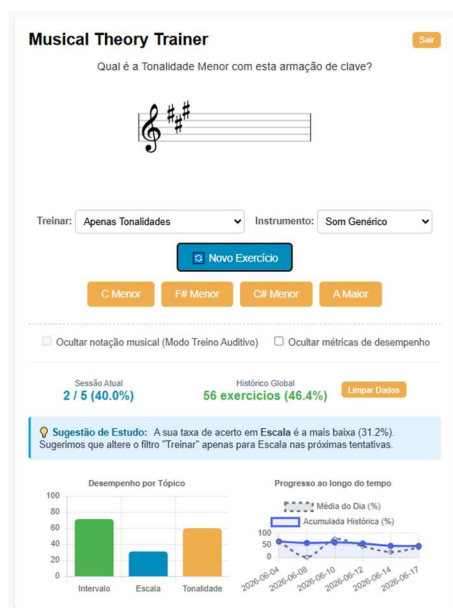


Figura 13 - Exemplo de exercício para identificação de tonalidades

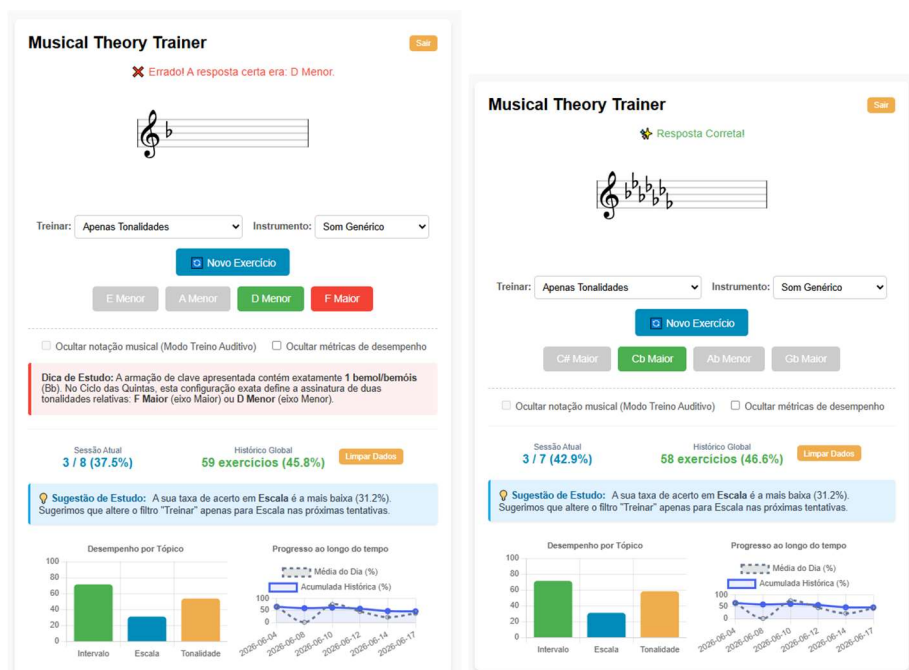


Figura 14 - Exemplo de exercício para identificação de tonalidades – resposta errada / correta



Figura 15 – Apresentação sem exibição da pauta e das métricas.

Capítulo 3 - Implementação

Este capítulo descreve a organização do trabalho em tarefas, a gestão de dependências e o estado final do desenvolvimento do projeto *Musical Theory Trainer* que culminou na sua conclusão e alojamento em ambiente de produção.

3.1. ESTRUTURA DE DECOMPOSIÇÃO DO TRABALHO (WBS)

Para garantir o cumprimento dos requisitos definidos (*MoSCoW* - *Must-have*, *Should-have*, *Could-have*, *Won't-have*), o projeto foi decomposto em quatro eixos principais de tarefas:

- **Gestão e Planeamento:** Redação da proposta, levantamento de requisitos, gestão de riscos e planeamento de arquitetura.
- **backend (Núcleo Lógico):** Implementação da API REST, motor de geração procedimental musical, configuração da base de dados *deploy* no *Render* e testes unitários automatizados.
- **Frontend (Interface e Experiência):** Criação da SPA, integração das bibliotecas de visualização (*VexFlow*) e áudio (*Tone.js*), desenho da UI/UX e implementação do *dashboard* detalhado (*Chart.js*).
- **Documentação e Relatórios:** Manutenção do diário de bordo (*Changelog*), redação de ADRs (*Architecture Decision Records*) e elaboração dos relatórios (intermédio e final).

3.2. PRECEDÊNCIAS E DISTRIBUIÇÃO DE TAREFAS

Tratando-se de um projeto individual, todas as tarefas foram executadas pelo utilizador. A ordem de implementação obedece à seguinte lógica de dependências:

1. **Motor Lógico e Validação:** O motor de lógica musical (*music_logic.py*) teve de ser desenvolvido antes da criação das rotas da API, para garantir a correção teórica absoluta dos dados gerados. Posteriormente foi validado através de testes unitários automatizados (*pytest*).
2. **backend e Persistência:** A estrutura da API (FastAPI) e os modelos de dados (*SQLite* e *SQLAlchemy*) foram finalizados antes do desenvolvimento do *Frontend*, permitindo que a interface consumisse dados reais. A implementação da Autenticação e cifra de credenciais foi efetuada posteriormente.
3. **Interface, Gráficos e Áudio:** A integração da notação vetorial (*VexFlow*) e do áudio (*Tone.js*) foi a tarefa técnica que fechou o núcleo (*core*), uma vez que a síntese sonora dependia da interação física do utilizador com os elementos já desenhados no ecrã para contornar as políticas de segurança dos *browsers*.
4. **Métricas e Sugestões de Estudo:** A implementação do *Dashboard* analítico (*Chart.js*) e do módulo de sugestões de estudo avançou apenas após a estabilização do ciclo de gravação de tentativas na base de dados, uma vez que dependia de um histórico real para gerar os gráficos.
5. **Alojamento (Deploy):** A última tarefa do projeto consistiu na transição do ambiente de desenvolvimento local para produção, alojando a aplicação e a respetiva documentação interativa na plataforma *cloud Render*.

3.3. CALENDARIZAÇÃO E PROGRESSO

O calendário previsto foi respeitado. O Produto Mínimo Viável (MVP) foi totalmente implementado, testado e em produção.

Tabela 3 - Calendarização e estado das tarefas

Semanas	Tarefa / Marco	Estado
Sem. 1-2	Proposta e Configuração do Repositório	Concluído
Sem. 3-4	Arquitetura C4, Requisitos e Setup API/BD	Concluído
Sem. 5-6	Lógica de Intervalos/Escalas e Integração VexFlow	Concluído
Sem. 7	Demo Interna e Consolidação de Código	Concluído
Sem. 8	Entrega do Relatório Intermédio	Concluído

Sem. 9-12	Implementação de Tonalidades e do Motor de justificações pedagógicas. Autenticação (SQLite), <i>deploy</i> no <i>Render</i> e Testes Funcionais	Concluído
Sem. 13-14	Treino Auditivo, Sugestões de estudo no Dashboard, Testes Unitários (pytest)	Concluído
Sem. 15-16	Relatório Final e Defesa Pública	Em Curso

3.4.RELATÓRIO DE ESTADO DA IMPLEMENTAÇÃO

O Produto Mínimo Viável está totalmente concluído. O estado de implementação dos Requisitos Funcionais (*Must-Have*) é o seguinte:

- **RF01 (Geração de Exercícios):** Concluído. O motor gera procedimentalmente exercícios de Intervalos, Escalas (incluindo Modos Gregos) e Tonalidades, garantindo a ortografia diatônica correta, sendo de relevar que no caso dos exercícios de Tonalidades apenas são geradas as armações de clave, não sendo geradas sequências de notas.
- **RF02 (Representação da Pauta):** Concluído. Integração bem-sucedida com *VexFlow*, incluindo gestão e desenho correto dos acidentes musicais na pauta (sustenidos, bemóis e bequadros).
- **RF03 (Reprodução do áudio):** Concluído. Sintetizador implementado via *Tone.js*, com reprodução das notas geradas. Foi também disponibilizada a possibilidade de selecionar timbres diferenciados.
- **RF04 (Opções de Escolha Múltipla):** Concluído. O sistema apresenta opções geradas pelo *backend* que funcionam como distratores, mapeando os erros mais frequentes dos utilizadores para conferir valor diagnóstico à resposta.
- **RF05 (Bloqueio após Submissão):** Concluído. Após a seleção de uma opção, todos os botões de resposta são desativados no *Frontend*, impedindo múltiplas submissões.
- **RF06/RF07 (Avaliação e Feedback Visual):** Concluído. A validação é comunicada visualmente. Em caso de erro, o sistema gera e exibe uma justificação teórica fundamentada nas notas exatas do exercício em causa.
- **RF08/RF09 (Persistência e Dashboard):** Concluído. O *dashboard* foi expandido para incluir métricas de sessão e globais, gráficos de desempenho com a biblioteca *Chart.js* e um "módulo de sugestões de estudo" que aconselha áreas de estudo com base nas maiores taxas de erro.

Importa destacar a superação da principal limitação técnica reportada no relatório intermédio. O requisito RF10 (*Should-Have*), relativo à Autenticação Básica, foi promovido a "*Must-Have*" e implementado com sucesso. A aplicação suporta agora múltiplos utilizadores, garantindo a encriptação de palavras-passe via *hashlib* e o

isolamento seguro do histórico de métricas para cada perfil, mantendo paralelamente a possibilidade de uso livre através de um Modo "Convidado".

Capítulo 4 - Testes

A estratégia de testes do *Musical Theory Trainer* foi dividida em duas etapas complementares para garantir a robustez do Produto Mínimo Viável (MVP): testes unitários automatizados ao motor lógico (*backend*) e testes funcionais manuais de aceitação à interface gráfica (*Frontend*).

4.1. TESTES UNITÁRIOS AUTOMATIZADOS (MOTOR LÓGICO)

A correção ortográfica e teórica é o requisito mais crítico de um sistema de tutoria musical. Para garantir que o motor procedimental não gera "enarmonias absurdas" ou escalas inválidas, foi implementada uma bateria de testes unitários automatizados com recurso à *framework* *pytest*.

A execução dos testes decorreu com 100% de sucesso, cobrindo os seguintes aspetos:

- **Teste de Ortografia Diatónica** (`test_obter_nota_ortografica...`): Garante que o motor distingue corretamente notas com o mesmo som, mas escritas de forma diferente (ex: Fá Sustenido vs. Sol Bemol), processando corretamente os identificadores para o *VexFlow* e para o *Tone.js*.
- **Teste de Intervalos** (`test_gerar_intervalo_aleatorio_estrutura`): Assegura que o motor gera exatamente duas notas na mesma oitava, acompanhadas por 4 opções de resposta (sendo apenas uma a correta) e uma justificação teórica em formato texto.
- **Teste de Escalas** (`test_gerar_escala_aleatoria_estrutura`): Valida que o construtor iterativo de intervalos gera escalas estruturadas com exatamente 8 notas (uma oitava completa), incluindo a validação de modos gregos.
- **Teste de Tonalidades** (`test_gerar_exercicio_tonalidade_estrutura`): Garante que as armações de clave geradas respeitam os limites matemáticos e musicais do Ciclo das Quintas (entre -7 bemóis e +7 sustenidos).

```

===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.4.1, pluggy-1.6.0 -- C:\Users\jopaf\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_gb25n2kfra8p0\python.exe
rootdir: C:\Users\jopaf\OneDrive - UAB\UAB\Ano Letivo 2025-26\2º Semestre\Projeto de Engenharia Informatica\VTI\PEI-MIA\src\backend
plugins: anyio-3.7.1, cov-6.2.1
collected 4 items

test_music_logic.py::test_obter_nota_ortografica_diferenciador_enarmônico PASSED [ 25%]
test_music_logic.py::test gerar Intervalo aleatório estrutura PASSED [ 50%]
test_music_logic.py::test gerar escala aleatoria estrutura PASSED [ 75%]
test_music_logic.py::test gerar exercicio tonalidade estrutura PASSED [100%]

===== 4 passed in 0.28s =====

```

Figura 16 – Resultado dos testes unitários automatizados⁷

4.2. TESTES FUNCIONAIS E DE INTERFACE

Para testar o comportamento da *Single Page Application* (SPA) e a integração com o *backend*, foram planeados e executados os cenários de teste evidenciados na tabela seguinte.

Tabela 4 – Cenários para testes de funcionais e de interface

ID	Cenário de Teste	Ações	Resultado Esperado
CTo1	Registo com Sucesso	Preencher "Utilizador" e "Password" novos e clicar em "Criar Conta".	Transição para o ecrã principal; <i>Dashboard</i> inicializado a zeros.
CTo2	Validação de Registo Duplicado	Tentar criar uma conta com um nome de utilizador que já existe.	Mensagem de erro vermelha: "Este nome de utilizador já está em uso."
CTo3	Login Inválido	Inserir password incorreta num utilizador existente e clicar em "Entrar".	Mensagem de erro vermelha: "Credenciais inválidas."
CTo4	Geração de Exercício	Escolher um filtro (ex: "Escala") e clicar em "Novo Exercício".	Pauta desenhada com a escala; 3 (escalas) ou 4 botões de opção gerados.
CTo5	Resposta Correta	Clicar no botão correspondente à resposta certa.	Botão fica verde; Mensagem "🎉 Resposta Correta!"; Taxa de acerto atualiza no <i>Dashboard</i> .
CTo6	Resposta Errada	Clicar num botão com a resposta incorreta.	Botão clicado a vermelho; Botão certo a verde; Caixa de justificação teórica é exibida.
CTo7	Funcionalidade Áudio	Num exercício gerado, selecionar um Timbre e clicar em "Tocar".	Reprodução sonora multitimbre das notas visíveis na pauta.

⁷ Após execução na pasta *backend* do *pytest* com a seguinte sintaxe:
 pytest test_music_logic.py -v -p no:cacheprovider

CTo8	Persistência de Dados	Clicar em "Sair", voltar a fazer Login com a mesma conta.	Ecrã principal carrega; Dashboard recupera a percentagem exata de acertos da sessão anterior.
CTo9	Sugestões de estudo	Falhar intencionalmente múltiplas vezes numa categoria específica (ex: Escalas).	O painel de sugestões de estudo evidencia a fraqueza e sugere o foco de estudo.

4.2.1. CTo1: Registo de Novo Utilizador com Sucesso

Este teste visa validar o processo de registo, garantindo a correta inserção de dados no *SQLite* e a transição da interface. Acionando o botão "Criar Conta", despoleta-se um pedido HTTP POST. A API recebe o pedido, aplica a geração de *hash SHA-256* e regista o novo utilizador através do *SQLAlchemy*. O servidor devolve um estado de sucesso. O *JavaScript* armazena o identificador e altera a visibilidade do *DOM*⁸, inicializando o *Dashboard* com zero tentativas.

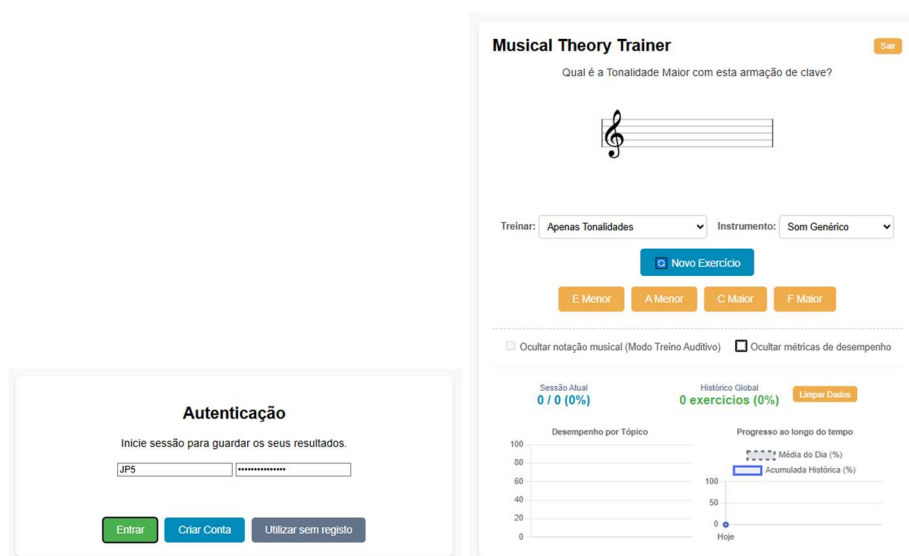


Figura 17 - Registo preenchido – Novo utilizador / métricas a zeros

⁸ *Document Object Model* - API criada pelos browsers que transforma o documento HTML numa árvore de nós/objetos permitindo que linguagens como o JavaScript interajam com a página, alterando a sua estrutura, estilo ou conteúdo em tempo real.

4.2.2. CT02: Validação de Registo Duplicado

Pretende-se verificar a integridade da base de dados. Inserindo um nome de utilizador já existente o sistema deteta a duplicação na consulta do ORM⁹ e o fluxo interrompe o registo. O *frontend* processa a rejeição exibindo a mensagem "Este nome de utilizador já está em uso", impedindo a sobreposição de métricas de estudo entre utilizadores distintos.



The screenshot shows a web form titled "Autenticação". Below the title is the instruction "Inicie sessão para guardar os seus resultados." There are two input fields: the first contains the text "JP5" and the second is filled with dots. Below the fields, a red error message reads "Este nome de utilizador já está em uso." At the bottom, there are three buttons: "Entrar" (green), "Criar Conta" (blue), and "Utilizar sem registo" (grey).

Figura 18 – Tentativa de registo com nome de utilizador já existente

4.2.3. CT03: Validação de Login Inválido

Com este teste assegura-se que o motor de verificação rejeita acessos indevidos. A password submetida sofre a transformação algorítmica (*SHA-256*)¹⁰ e, falhando a correspondência com o *hash* da base de dados, a operação é bloqueada com a mensagem "Credenciais inválidas" exibida na SPA.



The screenshot shows the same "Autenticação" form as in Figure 18. The input fields contain "JP5" and a password. Below the fields, a red error message reads "Credenciais inválidas." The buttons "Entrar", "Criar Conta", and "Utilizar sem registo" are still present at the bottom.

Figura 19 – Tentativa de autenticação com password incorreta

⁹ *Object-Relational Mapping* - ponte que liga o código (*Python*) à base de dados (*SQLite*).

¹⁰ *Secure Hash Algorithm 256-bit* - função matemática de criptografia que transforma texto numa sequência alfanumérica única e fixa de 64 caracteres.

4.2.4. CTo4: Geração de Exercício

Selecionando um filtro (ex: Escalas) e pressionando "Novo Exercício", o motor devolve com sucesso as propriedades musicais e os distratores inteligentes. A biblioteca *VexFlow* formata o SVG, os 4 botões são criados no contentor, e o sintetizador fica alocado na memória global.

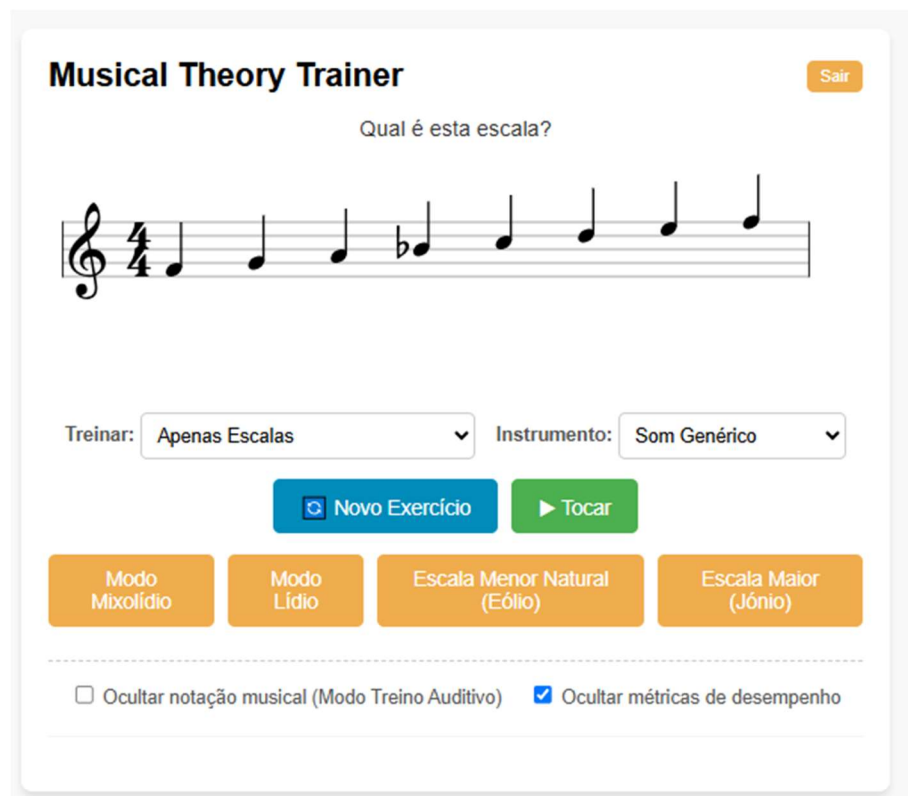


Figura 20 – Pauta desenhada corretamente. Quatro opções de resposta.

4.2.5. CTo5 (Resposta Correta) e CTo6 (Resposta Errada)

Avalia-se o funcionamento da tutoria inteligente. Clicando numa resposta certa (CTo5), a interface adota a cor verde, atualizando os gráficos em tempo real. Num cenário de falha (CTo6), o botão submetido fica vermelho e a solução exata surge realçada a verde. Cumprindo as regras de tutoria, é imediatamente exibida uma caixa de justificação pedagógica formulada a partir das notas desenhadas no ecrã. Os botões são bloqueados após a primeira submissão para evitar corrupção de métricas (RFo5).

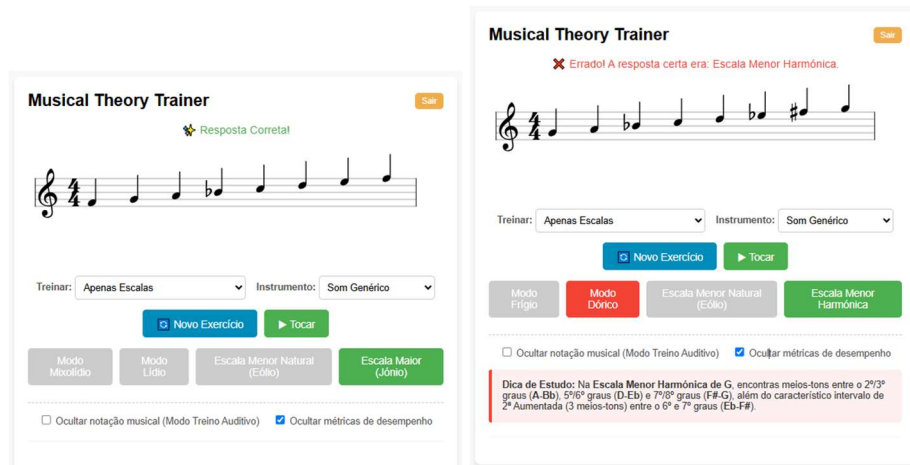


Figura 21 –Resposta correta e incorreta, com apresentação da justificação pedagógica.

4.2.6. CT07: Funcionalidade Áudio e CT08: Persistência

Acionando o botão "Tocar", o comando *Tone.start()* desbloqueia o contexto de áudio em concordância com as regras de segurança dos browsers. O algoritmo itera sobre o *array* melódico emitindo as frequências exatas. Reiniciando a sessão (CT08), o *dashboard* demonstra a recuperação do histórico de desempenho global gravado persistentemente no SQLite.

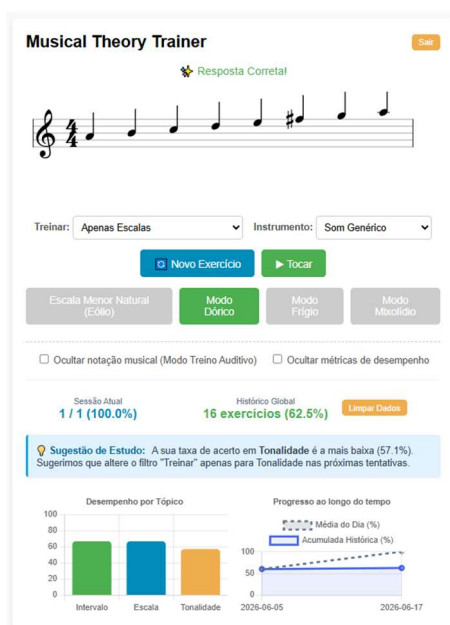


Figura 22 - Gráficos de recuperação de sessão histórica (utilizador JP2)

4.2.7. CT09: Sugestões de estudo

Ao gerar assimetrias no histórico (ex: elevadas taxas de erro na identificação de Modos), o script do *Dashboard* identifica automaticamente a área mais fraca. A SPA exibe dinamicamente as sugestões de estudo a azul, sugerindo ao utilizador a aplicação de um filtro específico para correção do défice visualizado nos gráficos gerados pelo *Chart.js*.

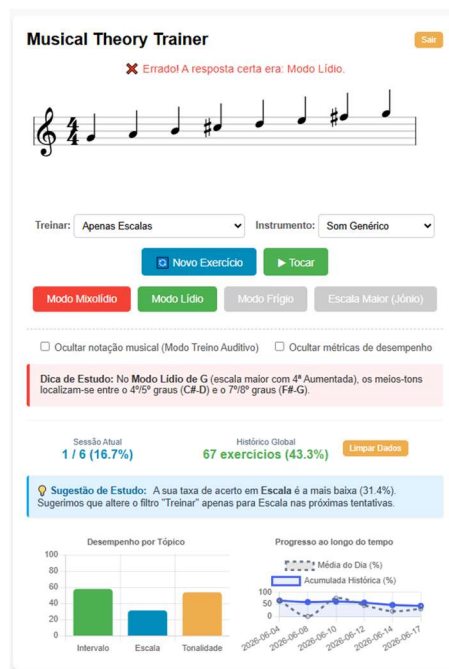


Figura 23 – Sugestão de estudo com base no tipo de exercício com menor taxa de acerto

Capítulo 5 - Conclusões

Este capítulo encerra o relatório com uma reflexão global sobre o trabalho desenvolvido, analisando o grau de cumprimento dos objetivos propostos. São também debatidas as limitações técnicas encontradas ao longo da conceção da arquitetura e traçadas as perspetivas de evolução futura do sistema.

5.1. REFLEXÃO SOBRE OS RESULTADOS

O balanço final do projeto *Musical Theory Trainer* é francamente positivo. O objetivo principal delineado na fase de proposta - criar uma plataforma web interativa para o treino autónomo da teoria musical - foi integralmente alcançado. O Produto Mínimo Viável (MVP) provou ser uma boa solução, superando mesmo o âmbito inicialmente previsto.

A nível tecnológico, a simbiose entre o *backend* (Python/FastAPI) e o *Frontend* (JavaScript/VexFlow/Tone.js) funcionou de forma harmoniosa. O sistema não depende de ficheiros áudio pré-gravados ou pautas estáticas, demonstrando que é possível gerar, em tempo real, pautas vetoriais (SVG) e frequências sonoras multitimbre diretamente no browser do utilizador.

A componente pedagógica, central neste projeto, foi consolidada com a implementação de um motor procedimental que não se limita a gerar exercícios aleatórios, mas que incorpora "distratores inteligentes" e um algoritmo de ortografia diatónica que assegura a precisão teórica. A adoção da *framework* *pytest* para a execução de testes unitários foi vital para atestar a estabilidade e coerência matemática deste motor. A superação das expectativas culminou na implementação de perfis de utilizador autenticados e num *Dashboard* equipado com um módulo de sugestões de estudo capaz de diagnosticar e orientar ativamente o estudo.

5.2. DIFICULDADES E LIMITAÇÕES

A realização deste projeto ocorreu em contexto académico, pelo que a sua amplitude e funcionalidade foram definidas pela carga horária (créditos ECTS) da Unidade Curricular Projeto de Engenharia Informática. Este projeto permitiu demonstrar a aplicação prática dos conhecimentos adquiridos ao longo do curso e propiciou aprendizagem através da resolução das dificuldades resultantes da inexperiência na área. Como o autor é trabalhador-estudante, a gestão do tempo constituiu um dos principais desafios não-técnicos do projeto. A necessidade de conciliar o desenvolvimento do software com as responsabilidades profissionais e familiares exigiu uma rigorosa priorização de requisitos. Esta realidade orientou os esforços para o núcleo essencial, assegurando a entrega atempada do Produto Mínimo Viável.

A construção de um sistema de avaliação procedimental exigiu compromissos ao nível da arquitetura, devidamente mapeados na Gestão de Riscos do projeto:

- **Delimitação do Espaço Musical:** Durante o desenvolvimento da lógica de geração de escalas e modos, verificou-se que a aleatoriedade cromática absoluta gerava "enarmonias absurdas" (ex: duplos sustenidos ou referências teóricas pouco convencionais). Para garantir a precisão pedagógica e a legibilidade, indispensáveis para utilizadores principiantes, foi necessário circunscrever o algoritmo a um subconjunto de 5 notas fundamentais. Esta limitação garante a usabilidade no MVP, mas restringe a exploração teórica avançada.
- **Validação *Client-Side* e Avaliação Formativa:** Para garantir que a aplicação seja rápida e parca na utilização de recursos no servidor, a validação das respostas foi delegada para o *Frontend* (JavaScript). Embora esta decisão seja ideal para um ambiente de "Avaliação Formativa" (estudo autónomo), impossibilita a utilização atual da plataforma para "Avaliação Sumativa" (exames formais), uma vez que utilizadores avançados poderiam inspecionar o código da página para descobrir a resposta correta enviada pela API.
- **Modelo de Avaliação e Segurança:** A decisão validar as respostas no *frontend* resulta de uma escolha deliberada para favorecer a fluidez da experiência, garantindo latência zero no *feedback* imediato. Reconhece-se que este modelo é inadequado para contextos de avaliação formal uma vez que o envio da resposta correta torna o sistema vulnerável à inspeção por utilizadores avançados. A distinção entre 'Treino' (onde o foco é a velocidade de resposta) e 'Exame' (onde a segurança é a prioridade) pode ser um dos eixos principais para o trabalho futuro, com a migração da lógica de validação para o *backend*.
- **Geração Procedimental Delimitada:** O motor de geração automática que sustenta a plataforma deve ser interpretado como um sistema de geração procedimental delimitado e não como uma cobertura exaustiva da teoria tonal. A restrição a um conjunto reduzido de cinco notas base e a casos musicais controlados foi essencial para garantir a precisão ortográfica das pautas geradas e a usabilidade para utilizadores iniciantes. Esta abordagem evita a geração de enarmonias complexas ou erros teóricos, também delimita o espaço musical disponível. A expansão deste espaço para uma cobertura total da teoria tonal, sem comprometer a legibilidade e a correção teórica, constitui um desafio técnico que requer evoluções algorítmicas significativas.
- **Fragilidades na Segurança e Autenticação:** A implementação da autenticação no projeto deve ser entendida como uma prova de conceito básica e não como uma solução de segurança definitiva. A utilização do algoritmo *SHA-256* diretamente para o armazenamento de passwords, sem o recurso a *salts*¹¹ ou a funções de *hashing* específicas para este fim (p.ex. *Bcrypt* ou *Argon2*), torna o sistema vulnerável a ataques de dicionário. Adicionalmente, a lógica de autorização confia nos identificadores (*userId*) armazenados no browser do

¹¹ *salt* - cadeia aleatória e única que adicionamos a uma senha antes de a transformar num hash. Por analogia, é como adicionar sal numa receita - muda o resultado final.

utilizador, o que permite a manipulação de pedidos para aceder ou apagar métricas alheias.

- **Alojamento e Latência *Cloud*:** O ciclo de desenvolvimento e a maioria dos testes funcionais e unitários decorreu em ambiente local. Embora o *deploy* para a plataforma *cloud Render* tenha sido bem-sucedido e a aplicação se encontre em produção, a natureza gratuita e partilhada desta infraestrutura de alojamento impõe limitações. O sistema pode ocasionalmente apresentar latência elevada no arranque ou anomalias na comunicação de rede ou não gestão dos dados armazenados que não são reproduzíveis no ambiente de desenvolvimento.

5.3. MELHORIAS E TRABALHO FUTURO

O projeto tem uma fundação técnica sólida que permite expansões futuras. Com base nas limitações identificadas, o trabalho futuro pode focar-se em três eixos:

1. ***Intelligent Tutoring System (ITS)* e Repetição Espaçada:** O *dashboard* descritivo atual pode evoluir para um verdadeiro motor de aprendizagem adaptativa. O sistema poderá memorizar não apenas as taxas de acerto gerais, mas o histórico cognitivo por cada nota ou intervalo específico, aplicando algoritmos de *Spaced Repetition* (ex: Sistema *Leitner* ¹²) para forçar a repetição dos erros mais frequentes.
2. **Modo de Exame (Validação *Server-Side*):** Para contornar a limitação de segurança na validação de respostas, pode ser criado um "Modo de Exame". Neste modo, o *backend* armazenará temporariamente a resposta certa em memória (ou numa base de dados), devolvendo a avaliação apenas após a submissão cega por parte do *Frontend*.
3. **Escalabilidade de Infraestrutura:** Com o crescimento da base de utilizadores, a atual base de dados embebida (*SQLite*) deve ser migrada para um Sistema de Gestão de Bases de Dados Relacionais mais robusto, como o *PostgreSQL*. Simultaneamente, o método atual de *hashing* de *passwords* (SHA-256) pode ser atualizado para o algoritmo *Bcrypt* (com injeção de *Salts*) para incrementar a segurança.

¹² método de estudo com cartões de memória (*flashcards*) baseado na repetição espaçada.

Bibliografia

Chart.js. (2026). *Chart.js - Simple yet flexible JavaScript charting for designers & developers*. Disponível em <https://www.chartjs.org/> (acedido em junho de 2026)

Desain, P., & Honing, H. (1992). *Music, Mind, and Machine: Studies in Computer Music, Music Cognition, and Artificial Intelligence*. Thesis Publishers.

FastAPI. (2026). FastAPI framework, high performance, easy to learn, fast to code, ready for production. Disponível em <https://fastapi.tiangolo.com/> (acedido em maio e junho de 2026)

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Hattie, J., & Timperley, H. (2007). The Power of Feedback. *Review of Educational Research*, 77(1), 81–112.

Kostka, S., Payne, D., & Almén, B. (2018). *Tonal harmony* (8.^a ed.). McGraw-Hill.

Product Plan. (2026). MoSCoW prioritization. Disponível em <https://www.productplan.com/glossary/moscow-prioritization/> (acedido em 3 de maio de 2026)

Pydantic. (2026). Pydantic - data validation using Python type hints. Disponível em <https://docs.pydantic.dev/> (acedido em maio e junho de 2026)

pytest. (2026). *pytest: helps you write better programs*. Disponível em <https://docs.pytest.org/> (acedido em junho de 2026)

Render. (2026). *Render - Cloud Hosting for Developers*. Disponível em <https://render.com/> (acedido em maio e junho de 2026)

Shute, V. J. (2008). Focus on Formative Feedback. *Review of Educational Research*, 78(1), 153–189.

Simon, S. (2023). The C4 model for visualising software architecture. Disponível em <https://c4model.com/> (acedido em 3 de maio de 2026)

SQLAlchemy. (2026). SQLAlchemy - the Python SQL toolkit and object relational mapper. Disponível em <https://www.sqlalchemy.org/> (acedido em 3 de maio de 2026)

SQLite. (2026). SQLite home page. Disponível em <https://www.sqlite.org/> (acedido em 3 de maio de 2026)

Togelius, J., Kastadjian, Y., Schecker, F., & Yannakakis, G. N. (2011). What is Procedural Content Generation? A Topology and Progress Report. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3), 172–186.

Tone.js. (2026). A Web Audio framework for making interactive music in the browser. Disponível em <https://tonejs.github.io/> (acedido em maio e junho de 2026)

VexFlow. (2026). VexFlow - music engraving in HTML5. Disponível em <https://vexflow.com/> (acedido em maio e junho de 2026)

Anexo I – Código desenvolvido

Reproduz-se o código do projeto, lembrando que se encontra integralmente disponível no GitHub: <https://github.com/jopaferreira/PEI-MTA>.

Releva-se que a totalidade do código-fonte está comentado, não só para refletir as normas exigidas na indústria de desenvolvimento de software, mas fundamentalmente para facilitar a leitura, a compreensão da lógica implementada e a futura manutenção ou escalabilidade do sistema.

FRONTEND - INDEX.HTML

```
<!DOCTYPE html>
<html lang="pt">

<head>
  <meta charset="UTF-8">
  <title>Musical Theory Trainer</title>
  <!-- Ícone vazio para suprimir o pedido automático do browser ao favicon.ico -->
  <link rel="icon" href="data:,">
  <!-- VexFlow 4.2.2 - notação musical em formato SVG -->
  <script src="https://cdn.jsdelivr.net/npm/vexflow@4.2.2/build/cjs/vexflow.js"></script>
  <!-- Tone.js 14.8.49 - Síntese de áudio no browser -->
  <script src="https://cdn.jsdelivr.net/npm/tone@14.8.49/Tone.js"></script>
  <!-- Chart.js - Geração dos gráficos do dashboard de métricas -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <link rel="stylesheet" href="styles.css">
</head>

<body>
  <!-- SPLASH SCREEN: Ecrã de carregamento animado (2s) enquanto as bibliotecas JS ficam prontas -->
  <div id="splash-screen">
    <div class="splash-content">
      <h1 class="splash-title">🎵 Musical Theory Trainer</h1>
      <p class="splash-subtitle">A afinar os instrumentos...</p>
      <div class="spinner"></div>
    </div>
  </div>

  <!-- ECRÃ DE AUTENTICAÇÃO: Visível após o splash se não houver sessão ativa -->
  <div id="login-screen" class="card" style="display: none;">
    <h2>Autenticação</h2>
    <p>Inicie sessão para guardar os seus resultados.</p>
    <div style="margin: 20px 0;">
      <input type="text" id="inputUser" placeholder="Nome de Utilizador" class="input-form">
      <input type="password" id="inputPass" placeholder="Password" class="input-form">
    </div>
    <!-- Área de mensagens de erro/validação da autenticação -->
    <p id="login-msg" style="color: #f44336; font-size: 14px; min-height: 20px;"></p>
    <div style="display: flex; gap: 10px; justify-content: center; flex-wrap: wrap; margin-top: 20px;">
      <button id="btnEntrar" class="btn-success">Entrar</button>
      <button id="btnRegistar" class="btn-primary">Criar Conta</button>
    </div>
    <!-- Modo convidado: acesso sem registo, sem persistência de resultados -->
```

```

        <button id="btnConvocado" style="background-color: #64748b; color: white;">Utilizar sem
registro</button>
    </div>
</div>

<!-- ECRÃ PRINCIPAL: Visível após autenticação ou acesso como convidado -->
<div id="app-screen" class="card" style="display: none;">

    <!-- Cabeçalho com título e botão de sair -->
    <div style="display: flex; justify-content: space-between; align-items: center;">
        <h2 style="margin: 0;">Musical Theory Trainer</h2>
        <button id="btnSair" class="btn-warning"
            style="padding: 5px 10px; font-size: 12px; margin-left: 10px;">Sair</button>
    </div>

    <!-- Mensagem dinâmica: indica o tipo de exercício ou o resultado da resposta -->
    <p id="status">A aguardar exercício...</p>

    <!-- Área de renderização SVG da pauta musical (VexFlow) -->
    <div id="pauta"></div>

    <!-- Filtros de exercício e seletor de timbre -->
    <div style="margin-bottom: 15px; display: flex; gap: 15px; justify-content: center; flex-wrap:
wrap;">
        <!-- Filtro: limita o tipo de exercício gerado -->
        <div>
            <label for="filtroExercicio" style="font-size: 14px; color:
#555;"><strong>Treinar:</strong></label>
            <select id="filtroExercicio"
                style="padding: 8px; border-radius: 5px; font-size: 14px; border: 1px solid #ccc;">
                <option value="Mistura">Intervalos, Escalas e Tonalidades</option>
                <option value="Intervalo">Apenas Intervalos</option>
                <option value="Escala">Apenas Escalas</option>
                <option value="Tonalidade">Apenas Tonalidades</option>
            </select>
        </div>
        <!-- Seletor de timbre: altera o tipo de sintetizador usado pelo Tone.js -->
        <div>
            <label for="seletorTimbre" style="font-size: 14px; color:
#555;"><strong>Instrumento:</strong></label>
            <select id="seletorTimbre"
                style="padding: 8px; border-radius: 5px; font-size: 14px; border: 1px solid #ccc;">
                <option value="sine">Som Genérico</option>
                <option value="triangle">Sintetizador Suave</option>
                <option value="square">Jogo Retro (8-bit)</option>
                <option value="sawtooth">Sintetizador Metálico</option>
                <option value="FMSynth">Piano Elétrico</option>
                <option value="AMSynth">Sopro</option>
                <option value="PluckSynth">Guitarra / Corda</option>
            </select>
        </div>
    </div>

    <!-- Botões principais de interação -->
    <div class="botoes">
        <button id="btnGerar" class="btn-primary"><img alt="play icon" data-bbox="510 638 525 648"/> Novo Exercício</button>
        <!-- Botão Tocar desativado até existir um exercício gerado; oculto nos exercícios de
Tonalidade -->
        <button id="btnTocar" class="btn-success disabled"><img alt="play icon" data-bbox="615 668 630 678"/> Tocar</button>
    </div>

    <!-- Botões de resposta gerados dinamicamente pelo app.js -->
    <div id="opcoes-resposta" class="botoes" style="margin-top: 15px;"></div>

    <!-- Opções de personalização da experiência de treino -->
    <div
        style="display: flex; gap: 20px; justify-content: center; margin-top: 25px; margin-bottom:
10px; flex-wrap: wrap; border-top: 1px dashed #ccc; padding-top: 15px;">
        <!-- Oculta a pauta para treino exclusivamente auditivo -->
        <label for="chkOcultarPauta" style="font-size: 14px; color: #555; cursor: pointer;">
            <input type="checkbox" id="chkOcultarPauta" /> Ocultar notação musical (Modo Treino
Auditivo)
        </label>
        <!-- Oculta o dashboard -->
        <label for="chkOcultarMetricas" style="font-size: 14px; color: #555; cursor: pointer;">
            <input type="checkbox" id="chkOcultarMetricas" /> Ocultar métricas de desempenho
        </label>
    </div>

    <!-- Caixa de justificação pedagógica: visível apenas após resposta errada -->
    <div id="explicacao-teorica"
        style="display: none; margin-top: 15px; padding: 15px; background-color: #fdf0f0; border-
left: 5px solid #f44336; text-align: left; font-size: 14px; color: #333; border-radius: 4px;">

```

```

    </div>

    <!-- DASHBOARD DE MÉTRICAS -->
    <div id="dashboard-area" style="margin-top: 20px; border-top: 2px solid #f1f5f9; padding-top:
20px;">
        <div id="dashboard-content">
            <!-- Indicadores numéricos: sessão atual e histórico global -->
            <div style="display: flex; justify-content: space-around; margin-bottom: 20px;">
                <div style="text-align: center;">
                    <span style="font-size: 12px; color: #64748b;">Sessão Atual</span><br>
                    <strong id="stat-sessao" style="font-size: 18px; color: #008CBA;">0 / 0
(0%)</strong>
                </div>
                <div id="bloco-historico" style="display: flex; gap: 20px; align-items: center;">
                    <div style="text-align: center;">
                        <span style="font-size: 12px; color: #64748b;">Histórico Global</span><br>
                        <strong id="stat-global" style="font-size: 18px; color: #4CAF50;">0 / 0
(0%)</strong>
                    </div>
                    <!-- Botão para apagar o histórico de tentativas do utilizador -->
                    <div style="display: flex; align-items: center;">
                        <button id="btnReset" class="btn-warning" style="padding: 5px 10px; font-
size: 12px;">Limpar
                            Dados</button>
                    </div>
                </div>
            </div>
            <!-- Recomendações pedagógicas -->
            <div id="painel-recomendacao"
                style="display: none; background-color: #e0f2fe; border-left: 4px solid #0ea5e9;
padding: 12px; margin-bottom: 20px; border-radius: 4px; text-align: left; box-shadow: 0 1px 3px
rgba(0,0,0,0.1);">
                <strong style="color: #0369a1; font-size: 14px;">💡 Sugestão de Estudo:</strong>
                <span id="texto-recomendacao" style="font-size: 14px; color: #334155; margin-left:
5px;"></span>
            </div>
            <!-- Gráficos Chart.js: desempenho por tipo de exercício e evolução temporal -->
            <div style="display: flex; gap: 15px; justify-content: center; height: 180px; margin-
top: 15px;">
                <div id="bloco-tipos" style="width: 45%;"><canvas id="graficoTipos"></canvas></div>
                <div id="bloco-evolucao" style="width: 45%;"><canvas
id="graficoEvolucao"></canvas></div>
            </div>
        </div>
    </div>

    <!-- Carrega o script principal da aplicação -->
    <script src="app.js"></script>
</body>
</html>

```

FRONTEND - APP.JS

```

// GESTÃO DA INTERFACE E ARRANQUE

// Remove Splash Screen 2 segundos após carregamento da página e decide qual ecrã mostrar
window.addEventListener('load', () => {
    const splashScreen = document.getElementById('splash-screen');
    if (splashScreen) {
        setTimeout(() => {
            splashScreen.classList.add('hidden');

            // Força o estado inicial: pauta visível e métricas ocultas
            configurarEstadoInicialOpcoes();

            // Após o splash, mostra o Login se o não estiver autenticado, ou a App se já tiver sessão
            // Verifica se o utilizador já tem ID (sessão ativa)
            if (localStorage.getItem("userId")) {
                const appScreen = document.getElementById("app-screen");
                if (appScreen) appScreen.style.display = "block";
                atualizarDashboard();
            }
        }, 2000);
    }
});

```

```

    } else {
        const loginScreen = document.getElementById("login-screen");
        if (loginScreen) loginScreen.style.display = "block";
    }
    }, 2000);
}
});

// Variáveis Globais
const API_URL = window.location.origin; // Produção (Render) => mesmo domínio
const VF = Vex.Flow;
const divPauta = document.getElementById("pauta");
let melodiaAtual = [];
let synth;
let exercicioAtual = null;
// Para as métricas melhoradas
let sessaoTotal = 0;
let sessaoCertas = 0;
let graficoTiposInstancia = null;
let graficoEvolucaoInstancia = null;
// Variável para indicar se o utilizador não pretende ter conta
let isGuest = false;
let sessaoTipos = { "Intervalo": [0, 0], "Escala": [0, 0], "Tonalidade": [0, 0] }; // Rastreo local

// Função auxiliar para garantir a conformidade dos estados iniciais das checkboxes e contentores
function configurarEstadoInicialOpcoes() {
    const chkOcultarPauta = document.getElementById("chkOcultarPauta");
    const chkOcultarMetricas = document.getElementById("chkOcultarMetricas");
    const dashboardContent = document.getElementById("dashboard-content");

    // 1. Notação Musical Visível por defeito (Treino auditivo desativado)
    if (chkOcultarPauta) chkOcultarPauta.checked = false;
    if (divPauta) divPauta.style.display = "flex";

    // 2. Painel Analítico Oculto por defeito (Métricas não visíveis no arranque)
    if (chkOcultarMetricas) chkOcultarMetricas.checked = true;
    if (dashboardContent) dashboardContent.style.display = "none";
}

// AUTENTICAÇÃO
// Função para gerir o login e o registo
async function gerirAutenticacao(rota) {
    const inputUser = document.getElementById("inputUser");
    const inputPass = document.getElementById("inputPass");
    const msgBox = document.getElementById("login-msg");
    // Validação dos campos
    if (!inputUser || !inputPass) return;
    // Verifica se ambos os campos estão preenchidos
    const user = inputUser.value;
    const pass = inputPass.value;
    // Se algum dos campos estiver vazio, mostra mensagem de erro e retorna
    if (!user || !pass) {
        if (msgBox) msgBox.innerText = "Preencha ambos os campos.";
        return;
    }
    // Envia os dados para a API e processa a resposta
    try {
        const resposta = await fetch(`${API_URL}/api/auth/${rota}`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username: user, password: pass })
        });

        const dados = await resposta.json();
        // Se a API retornar um erro, mostra a mensagem de erro
        if (dados.status === "erro") {
            if (msgBox) msgBox.innerText = dados.mensagem;
        } else {
            // Sucesso! Grava o ID, esconde o ecrã de login e mostra a aplicação
            localStorage.setItem("userId", dados.id);

            const loginScreen = document.getElementById("login-screen");
            const appScreen = document.getElementById("app-screen");
            // Limpa os campos e mensagens para a próxima vez que o utilizador se autenticar
            if (loginScreen) loginScreen.style.display = "none";
            if (appScreen) appScreen.style.display = "block";
            inputPass.value = "";
            if (msgBox) msgBox.innerText = "";

            // Garante o alinhamento das opções no login bem-sucedido
            configurarEstadoInicialOpcoes();

            // Atualiza o dashboard com os resultados do utilizador

```

```

        atualizarDashboard();
    }
} catch (error) {
    if (msgBox) msgBox.innerText = "Erro de ligação ao servidor.";
}
}

// Botão de Convidado - acesso sem conta e sem gravação de progresso
const btnConvidado = document.getElementById("btnConvidado");
if (btnConvidado) {
    btnConvidado.addEventListener("click", () => {
        isGuest = true;
        localStorage.removeItem("userId"); // Garante que não há sessão ativa

        document.getElementById("login-screen").style.display = "none";
        document.getElementById("app-screen").style.display = "block";

        // Oculta funcionalidades de utilizador registado
        document.getElementById("bloco-historico").style.display = "none";
        document.getElementById("bloco-evolucao").style.display = "none";
        document.getElementById("bloco-tipos").style.width = "100%"; // Expande o gráfico de sessão

        // Forçar estado limpo das checkboxes para o arranque da sessão convidada
        configurarEstadoInicialOpcoes();

        // Reinicia contadores locais
        sessaoTotal = 0;
        sessaoCertas = 0;
        sessaoTipos = { "Intervalo": [0, 0], "Escala": [0, 0], "Tonalidade": [0, 0] };

        atualizarDashboard();
    });
}

// Verifica se os botões existem antes de adicionar os event listeners para evitar erros em páginas onde
eles não estão presentes
const btnEntrar = document.getElementById("btnEntrar");
if (btnEntrar) {
    btnEntrar.addEventListener("click", () => gerirAutenticacao("login"));
}

const btnRegistar = document.getElementById("btnRegistar");
if (btnRegistar) {
    btnRegistar.addEventListener("click", () => gerirAutenticacao("registo"));
}

// Botão de Sair: Limpa a sessão e volta para o ecrã de login
const btnSair = document.getElementById("btnSair");
if (btnSair) {
    btnSair.addEventListener("click", () => {
        localStorage.removeItem("userId");
        const appScreen = document.getElementById("app-screen");
        const loginScreen = document.getElementById("login-screen");
        if (appScreen) appScreen.style.display = "none";
        if (loginScreen) loginScreen.style.display = "block";

        // Repõe interface e variáveis para o próximo utilizador
        sessaoTotal = 0;
        sessaoCertas = 0;
        isGuest = false;

        const blocoHistorico = document.getElementById("bloco-historico");
        const blocoEvolucao = document.getElementById("bloco-evolucao");
        const blocoTipos = document.getElementById("bloco-tipos");
        if (blocoHistorico) blocoHistorico.style.display = "flex";
        if (blocoEvolucao) blocoEvolucao.style.display = "block";
        if (blocoTipos) blocoTipos.style.width = "45%";

        configurarEstadoInicialOpcoes();
    });
}

// COMUNICAÇÃO COM A API E LÓGICA DE EXERCÍCIOS
const btnGerar = document.getElementById("btnGerar");
if (btnGerar) {
    btnGerar.addEventListener("click", async () => {
        const divOpcoes = document.getElementById("opcoes-resposta");
        const divExplicacao = document.getElementById("explicacao-teorica");
        const filtroExercicio = document.getElementById("filtroExercicio");
        // Limpa a pauta, opções e explicação para o novo exercício
        if (divOpcoes) divOpcoes.innerHTML = "";
    });
}

```

```

    if (divPauta) divPauta.innerHTML = "";
    if (divExplicacao) divExplicacao.style.display = "none";
    // Define o filtro selecionado ou usa "Mistura" como padrão
    const filtroSelecionado = filtroExercicio ? filtroExercicio.value : "Mistura";
    // Solicita um novo exercício com base no filtro selecionado
    try {
        const resposta = await fetch(`${API_URL}/api/exercicio/novo?filtro=${filtroSelecionado}`);
        const dados = await resposta.json();
        // Atualiza as variáveis globais com os dados do exercício recebido
        melodiaAtual = dados.notas || [];
        exercicioAtual = dados;
        // Atualiza o status do exercício e a visibilidade do botão de tocar em função do tipo de
exercício
        const status = document.getElementById("status");
        if (status) {
            status.innerText = dados.mensagem;
            status.style.color = "#333";
        }
        // Gestão do Botão Tocar e da Checkbox de Ocultar Pauta
        const btnTocar = document.getElementById("btnTocar");
        const chkOcultarPauta = document.getElementById("chkOcultarPauta");

        if (dados.tipo_exercicio === "Tonalidade") {
            if (btnTocar) btnTocar.style.display = "none";
            // Desativa a opção de ocultar pauta para Tonalidades (precisam do visual)
            if (chkOcultarPauta) {
                chkOcultarPauta.disabled = true;
                if (divPauta) divPauta.style.display = "flex"; // Força a visualização
            }
        } else {
            if (btnTocar) {
                btnTocar.style.display = "inline-block";
                btnTocar.disabled = false;
            }
            // Reativa a checkbox e aplica a visibilidade escolhida
            if (chkOcultarPauta) {
                chkOcultarPauta.disabled = false;
                if (divPauta) divPauta.style.display = chkOcultarPauta.checked ? "none" : "flex";
            }
        }
        // Desenha a pauta com as notas do exercício e cria os botões de resposta
        desenharPauta(dados);
        criarBotoesResposta(dados.opcoes, dados.detalhe);
        atualizarDashboard();
    } catch (e) {
        const status = document.getElementById("status");
        if (status) status.innerText = "Erro ao obter exercício do servidor.";
    }
    });
}

// Cria Botões de Resposta
function criarBotoesResposta(opcoes, respostaCerta) {
    const divOpcoes = document.getElementById("opcoes-resposta");
    if (!divOpcoes) return;

    divOpcoes.innerHTML = "";

    opcoes.forEach(opcao => {
        const btn = document.createElement("button");
        btn.innerText = opcao;
        btn.className = "btn-warning";

        btn.addEventListener("click", async () => {
            const acertou = (opcao === respostaCerta);

            const todosBotoes = divOpcoes.querySelectorAll("button");
            // Desativa todos os botões e atribui cores: verde para a resposta certa, vermelho para
errada
            todosBotoes.forEach(b => {
                b.disabled = true;
                if (b.innerText === respostaCerta) {
                    b.style.backgroundColor = "#4CAF50";
                } else if (b === btn && !acertou) {
                    b.style.backgroundColor = "#f44336";
                }
            });

            // Atualiza o status da resposta e mostra a explicação
            const status = document.getElementById("status");
            const divExplicacao = document.getElementById("explicacao-teorica");

```

```

        if (acertou) {
            if (status) {
                status.innerText = "🎯 Resposta Correta!";
                status.style.color = "#4CAF50";
            }
        } else {
            if (status) {
                status.innerText = `❌ Errado! A resposta certa era: ${respostaCerta}.`;
                status.style.color = "#f44336";
            }
            if (divExplicacao && exercicioAtual) {
                divExplicacao.innerHTML = `Dica de Estudo:</strong>`;
                divExplicacao.style.display = "block";
            }
        }
    }

    // Registo de métricas e gravação na BD
    if (exercicioAtual) {
        // Incrementa as variáveis da sessão (Modo Registrado e Convidado)
        sessaoTotal++;
        if (acertou) sessaoCertas++;

        // Rastreoio local por tipo de exercício
        sessaoTipos[exercicioAtual.tipo_exercicio][1]++;
        if (acertou) sessaoTipos[exercicioAtual.tipo_exercicio][0]++;

        // Só envia para a Base de Dados se NÃO for convidado (isGuest = false)
        if (!isGuest) {
            const userId = localStorage.getItem("userId");
            if (userId) {
                try {
                    await fetch(`${API_URL}/api/tentativas`, {
                        method: "POST",
                        headers: { "Content-Type": "application/json" },
                        body: JSON.stringify({
                            utilizador_id: parseInt(userId),
                            tipo_exercicio: exercicioAtual.tipo_exercicio,
                            detalhe: exercicioAtual.detalhe,
                            resposta_dada: opcao,
                            correta: acertou
                        })
                    });
                } catch (e) {
                    console.error("Erro ao guardar tentativa.");
                }
            }
        }

        // Atualiza os gráficos sempre (quer seja convidado ou não)
        atualizarDashboard();
    }
});

divOpcoes.appendChild(btn);
});
}

// Obtém e atualiza os resultados no Dashboard e Gráficos
async function atualizarDashboard() {
    // Atualiza o texto da Sessão Atual
    const statSessao = document.getElementById("stat-sessao");
    const taxaSessao = sessaoTotal > 0 ? ((sessaoCertas / sessaoTotal) * 100).toFixed(1) : 0;
    if (statSessao) statSessao.innerText = `${sessaoCertas} / ${sessaoTotal} (${taxaSessao}%`;

    if (graficoTiposInstancia) graficoTiposInstancia.destroy();
    const ctxTipos = document.getElementById('graficoTipos').getContext('2d');

    // utilizadores sem registo
    if (isGuest) {
        const dadosLocais = {};
        for (const [tipo, valores] of Object.entries(sessaoTipos)) {
            dadosLocais[tipo] = valores[1] > 0 ? ((valores[0] / valores[1]) * 100).toFixed(1) : 0;
        }

        graficoTiposInstancia = new Chart(ctxTipos, {
            type: 'bar',
            data: {
                labels: Object.keys(dadosLocais),
                datasets: [{

```

```

        label: '% de Acerto (Sessão Atual)',
        data: Object.values(dadosLocais),
        backgroundColor: ['#4CAF50', '#008CBA', '#f0ad4e'],
        borderRadius: 5
    }],
    options: { responsive: true, maintainAspectRatio: false, scales: { y: { beginAtZero: true,
max: 100 } }, plugins: { title: { display: true, text: 'Desempenho por Tópico (Sessão)' }, legend: {
display: false } } }
});

// Recomendações pedagógicas - Para utilizadores sem registo
atualizarRecomendadorPedagogico(dadosLocais);

return;
}

// utilizadores Registados
const userId = localStorage.getItem("userId");
if (!userId) return;

try {
    const resposta = await fetch(`${API_URL}/api/dashboard/${userId}`);
    const dados = await resposta.json();

    // Atualiza texto Global
    const statGlobal = document.getElementById("stat-global");
    if (statGlobal) statGlobal.innerHTML = `${dados.total} exercícios (${dados.taxa_global}%)`;

    // Elimina gráficos antigos antes de redesenhar para evitar sobreposição
    if (graficoEvolucaoInstancia) graficoEvolucaoInstancia.destroy();

    // Gráfico de Barras: Taxa desagregada por tipo de exercício
    graficoTiposInstancia = new Chart(ctxTipos, {
        type: 'bar',
        data: {
            labels: Object.keys(dados.por_tipo),
            datasets: [{
                label: '% de Acerto',
                data: Object.values(dados.por_tipo),
                backgroundColor: ['#4CAF50', '#008CBA', '#f0ad4e'],
                borderRadius: 5
            }
        ]
    },
    options: {
        responsive: true, maintainAspectRatio: false,
        scales: { y: { beginAtZero: true, max: 100 } },
        plugins: {
            title: { display: true, text: 'Desempenho por Tópico' },
            legend: { display: false }
        }
    }
});

// Gráfico de Linhas: Evolução Diária vs Acumulada
const ctxEvolucao = document.getElementById('graficoEvolucao').getContext('2d');
const datas = dados.evolucao_diaria.map(d => d.data);
const taxasDia = dados.evolucao_diaria.map(d => d.taxa_dia);
const taxasAcum = dados.evolucao_diaria.map(d => d.taxa_acumulada);

graficoEvolucaoInstancia = new Chart(ctxEvolucao, {
    type: 'line',
    data: {
        labels: datas.length > 0 ? datas : ['Hoje'],
        datasets: [
            {
                label: 'Média do Dia (%)',
                data: taxasDia.length > 0 ? taxasDia : [0],
                borderColor: '#64748b',
                borderDash: [5, 5],
                tension: 0.3,
                fill: false
            },
            {
                label: 'Acumulada Histórica (%)',
                data: taxasAcum.length > 0 ? taxasAcum : [0],
                borderColor: '#4361ee',
                borderWidth: 3,
                tension: 0.3,
                fill: true,
                backgroundColor: 'rgba(67, 97, 238, 0.1)'
            }
        ]
    }
});

```



```

    },
    options: {
      responsive: true, maintainAspectRatio: false,
      scales: { y: { beginAtZero: true, max: 100 } },
      plugins: {
        title: { display: true, text: 'Progresso ao longo do tempo' }
      }
    }
  });

  // Recomendações pedagógicas - Para utilizadores registados
  atualizarRecomendadorPedagogico(dados.por_tipo);

} catch (e) {
  console.error("Erro ao carregar métricas analíticas.");
}
}

// Configuração do Hard Reset
const btnReset = document.getElementById("btnReset");
if (btnReset) {
  btnReset.addEventListener("click", async () => {
    if (confirm("Tem a certeza que deseja apagar todo o seu histórico? Esta ação é irreversível.")) {
      const userId = localStorage.getItem("userId");
      try {
        await fetch(`${API_URL}/api/utilizadores/${userId}/reset`, { method: "DELETE" });
        sessaoTotal = 0;
        sessaoCertas = 0;
        atualizarDashboard();
      } catch (e) {
        console.error("Erro ao reiniciar dados.");
      }
    }
  });
}

// Desenho da Pauta Musical (VexFlow)
function desenharPauta(dados) {
  if (!divPauta) return;
  divPauta.innerHTML = "";
  // Define a largura da pauta com base no número de notas: mais notas exigem uma pauta mais longa
  const larguraPauta = (dados.notas && dados.notas.length > 2) ? 550 : 250;
  // Cria o renderer do VexFlow e a pauta, adicionando a clave de sol e, se for um exercício de
  tonalidade, a armação de clave correspondente
  const renderer = new VF.Renderer(divPauta, VF.Renderer.Backends.SVG);
  renderer.resize(larguraPauta + 50, 150);
  const context = renderer.getContext();
  const stave = new VF.Stave(10, 0, larguraPauta).addClef("treble");
  // Para exercícios de tonalidade, em vez de desenhar notas, desenha a armação de clave
  correspondente ao número de acidentes
  if (dados.tipo_exercicio === "Tonalidade") {
    // Atribui número de acidentes para a armação de clave
    const vexflowKeys = {
      "-7": "Cb", "-6": "Gb", "-5": "Db", "-4": "Ab", "-3": "Eb", "-2": "Bb", "-1": "F",
      "0": "C",
      "1": "G", "2": "D", "3": "A", "4": "E", "5": "B", "6": "F#", "7": "C#"
    };
    const chave = vexflowKeys[dados.num_acidentes.toString()];
    stave.addKeySignature(chave);
    stave.setContext(context).draw();
    return;
  }
  // Para exercícios de leitura de notas, desenha a pauta com as notas fornecidas pela API
  stave.addTimeSignature("4/4").setContext(context).draw();
  // Converte as notas do formato da API para o formato do VexFlow, adicionando acidentes quando
  necessário
  const vexNotes = dados.notas.map(nota => {
    const staveNote = new VF.StaveNote({ keys: [nota.vexflow], duration: "q" });
    let sinal = null;
    if (nota.vexflow.includes("#")) {
      sinal = new VF.Accidental("#");
    } else if (nota.vexflow.charAt(1) === "b") {
      sinal = new VF.Accidental("b");
    }

    if (sinal) {
      try { staveNote.addModifier(sinal, 0); } catch (e) { staveNote.addModifier(0, sinal); }
    }
    return staveNote;
  });
}

```

```

const voice = new VF.Voice({ num_beats: dados.notas.length, beat_value: 4 }).addTickables(vexNotes);
new VF.Formatter().joinVoices([voice]).format([voice], larguraPauta - 50);
voice.draw(context, stave);
}

// Reprodução de Áudio (Tone.js) com seleção de timbre
const btnTocar = document.getElementById("btnTocar");
if (btnTocar) {
  btnTocar.addEventListener("click", async () => {
    if (melodiaAtual.length === 0) return;
    await Tone.start();

    const seletorTimbre = document.getElementById("seletorTimbre");
    const tipoTimbre = seletorTimbre ? seletorTimbre.value : "Synth";

    // Limpa o sintetizador anterior para evitar sobreposição
    if (synth) {
      synth.dispose();
    }

    // Novo sintetizador com base na escolha do utilizador para o timbre
    if (tipoTimbre === "FMSynth") {
      synth = new Tone.FMSynth().toDestination();
    } else if (tipoTimbre === "AMSynth") {
      synth = new Tone.AMSynth().toDestination();
    } else if (tipoTimbre === "PluckSynth") {
      synth = new Tone.PluckSynth().toDestination();
    } else {
      // Lida automaticamente com: "sine", "triangle", "square" e "sawtooth"
      synth = new Tone.Synth({ oscillator: { type: tipoTimbre } }).toDestination();
    }

    const tempoAtual = Tone.now();
    melodiaAtual.forEach((nota, index) => {
      synth.triggerAttackRelease(nota.tone, "4n", tempoAtual + (index * 0.5));
    });
  });
}

// Ocultar/Mostrar Pauta (Treino Auditivo)
const chkOcultarPauta = document.getElementById("chkOcultarPauta");
if (chkOcultarPauta && divPauta) {
  chkOcultarPauta.addEventListener("change", (e) => {
    // Bloqueia a ação se for um exercício de Tonalidade
    if (exercicioAtual && exercicioAtual.tipo_exercicio === "Tonalidade") {
      divPauta.style.display = "flex";
      return;
    }
    // Alterna entre display 'none' (escondido) e 'flex'
    divPauta.style.display = e.target.checked ? "none" : "flex";
  });
}

// Ocultar/Mostrar Métricas
const chkOcultarMetricas = document.getElementById("chkOcultarMetricas");
const dashboardContent = document.getElementById("dashboard-content");

if (chkOcultarMetricas && dashboardContent) {
  chkOcultarMetricas.addEventListener("change", (e) => {
    // Alterna entre ocultar e mostrar as métricas
    dashboardContent.style.display = e.target.checked ? "none" : "block";
  });
}

// Função para gerar a sugestões de estudo com base na área mais fraca
function atualizarRecomendadorPedagogico(dadosTipos) {
  const painel = document.getElementById("painel-recomendacao");
  const texto = document.getElementById("texto-recomendacao");
  if (!painel || !texto) return;

  const topicos = Object.keys(dadosTipos);
  if (topicos.length > 0) {
    let piorTopico = topicos[0];
    let piorTaxa = parseFloat(dadosTipos[piorTopico]);

    // Encontra o tópico com a percentagem de acerto mais baixa
    for (const topico of topicos) {
      const taxaAtual = parseFloat(dadosTipos[topico]);
      if (taxaAtual < piorTaxa) {
        piorTaxa = taxaAtual;
      }
    }
  }
}

```

```

        piorTopico = topico;
    }
}

painel.style.display = "block";

// Regras de diagnóstico pedagógico
if (piorTaxa >= 80) {
    texto.innerHTML = `Excelente desempenho geral! Continue a treinar no modo
<strong>Mistura</strong> para manter a fluidez de raciocínio visual e auditivo.`;
} else {
    texto.innerHTML = `A sua taxa de acerto em <strong>${piorTopico}</strong> é a mais baixa
(${piorTaxa.toFixed(1)}%). Sugerimos que altere o filtro "Treinar" apenas para ${piorTopico} nas
próximas tentativas.`;
}
} else {
    painel.style.display = "none";
}
}
}

```

FRONTEND - STYLES.CSS

```

/* ESTILOS GERAIS */

/* Configuração global: centra o texto, define o tipo de letra e a cor do fundo */
body {
    font-family: sans-serif;
    text-align: center;
    margin-top: 40px;
    background-color: #f9f9f9;
}

/* Contendor principal da interface (estilo "cartão" branco flutuante com sombra) */
.card {
    background: white;
    max-width: 600px;
    margin: 0 auto;
    /* Centra o cartão horizontalmente */
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

/* Desenho da pauta musical (VexFlow) */
#pauta {
    display: flex;
    justify-content: center;
    /* Garante que a pauta fica ao centro */
    height: 160px;
    margin: 20px 0;
}

/* Contendor que agrupa os botões de ação na horizontal com espaçamento */
.botoes {
    gap: 10px;
    display: flex;
    justify-content: center;
    margin-top: 10px;
}

/* Estilo compartilhado por todos os botões */
button {
    padding: 10px 20px;
    font-size: 15px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

```

```

/* Cores específicas para cada tipo de ação */
.btn-primary {
    background-color: #008CBA;
    color: white;
}

/* Azul para ações neutras (Gerar) */
.btn-success {
    background-color: #4CAF50;
    color: white;
}

/* Verde para ações positivas (Tocar) */
.btn-warning {
    background-color: #f0ad4e;
    color: white;
}

/* Laranja para ações de submissão (Responder) */

/* Estilo visual para indicar que o utilizador não pode clicar no botão no momento */
button:disabled {
    background-color: #ccc;
    cursor: not-allowed;
}

/* ESTILOS DO SPLASH SCREEN */

/* Sobreposição de ecrã inteiro */
#splash-screen {
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background: linear-gradient(135deg, #08427b 0%, #1168bd 100%);
    /* Fundo azul */
    color: white;
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 9999;
    /* Garante que fica por cima de tudo */
    transition: opacity 0.8s ease-out, visibility 0.8s ease-out;
    /* Transição suave no desaparecimento */
}

/* Animação da entrada do texto inicial - de baixo para cima */
.splash-content {
    text-align: center;
    animation: fadeIn 1s ease-out;
}

/* Tipografia principal do Splash Screen */
.splash-title {
    font-size: 2.8rem;
    margin-bottom: 10px;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    text-shadow: 0 4px 6px rgba(0, 0, 0, 0.2);
}

.splash-subtitle {
    font-size: 1.2rem;
    opacity: 0.85;
    margin-bottom: 40px;
}

/* Ícone rotativo */
.spinner {
    width: 50px;
    height: 50px;
    margin: 0 auto;
    border: 4px solid rgba(255, 255, 255, 0.2);
    /* Borda transparente */
    border-top: 4px solid white;
    /* Apenas um bordo pintado para criar a ilusão de movimento */
    border-radius: 50%;
    /* Torna o quadrado num círculo perfeito */
    animation: spin 1s linear infinite;
    /* Roda infinitamente */
}

/* ANIMAÇÕES CSS */

```

```

/* Define a rotação de 0 a 360 graus para o Spinner */
@keyframes spin {
  0% {
    transform: rotate(0deg);
  }

  100% {
    transform: rotate(360deg);
  }
}

/* Define o fade in levantando os elementos ligeiramente */
@keyframes fadeIn {
  0% {
    opacity: 0;
    transform: translateY(20px);
  }

  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

/* ESTADOS CONTROLADOS PELO JAVASCRIPT */

/* Classe injetada pelo app.js (após 2s) para fazer o ecrã desaparecer */
#splash-screen.hidden {
  opacity: 0;
  visibility: hidden;
}

```

BACKEND – MAIN.PY

```

# Motor principal - Geração Musical e de Gravação de Resultados

# Bibliotecas - FastAPI e validação de dados
from fastapi import FastAPI, Depends
from fastapi.middleware.cors import CORSMiddleware
# Para servir o frontend (HTML/JS) diretamente do backend
from fastapi.staticfiles import StaticFiles
# Para definir os modelos de dados usados pela API
from pydantic import BaseModel

# Bibliotecas - Base de Dados e utilitários
from sqlalchemy.orm import Session, sessionmaker
import random
# Para navegar no sistema de ficheiros
import os
# Para criar o hash da password (segurança)
import hashlib

# Importa modelo da Base de Dados
from models import engine, Tentativa, Utilizador, Base
# Importa função para gerar exercício de tonalidade
from music_logic import gerar_intervalo_aleatorio, gerar_escala_aleatoria, gerar_exercicio_tonalidade

# Fundamental para o RENDER: Cria as tabelas na base de dados (se ainda não existirem)
Base.metadata.create_all(bind=engine)

# Sessões para comunicação com a Base de Dados - SQLite
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Inicializa aplicação
app = FastAPI()

# Configuração CORS: Permite que o Frontend (HTML/JS) comunique com a API sem bloqueios
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

# Abre e fecha a ligação à base de dados para cada pedido HTTP
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Valida os dados enviados pelo Frontend
class TentativaCreate(BaseModel):
    utilizador_id: int
    tipo_exercicio: str
    detalhe: str
    resposta_dada: str
    correta: bool

# LÓGICA DE AUTENTICAÇÃO

# Transforma a password num hash SHA-256
def encriptar_password(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()

# Registo de Utilizador - Verifica se o username já existe e guarda o hash da password
@app.post("/api/auth/registo")
def registar_utilizador(dados: dict, db: Session = Depends(get_db)):
    username = dados.get("username")
    password = dados.get("password")

    if not username or not password:
        return {"status": "erro", "mensagem": "Username e password obrigatórios."}

    # Verifica se o utilizador já existe
    existe = db.query(Utilizador).filter(Utilizador.username == username).first()
    if existe:
        return {"status": "erro", "mensagem": "Este nome de utilizador já está em uso."}
    # Cria o novo utilizador com a password encriptada e guarda na base de dados
    novo_user = Utilizador(username=username, password_hash=encriptar_password(password))
    db.add(novo_user)
    db.commit()
    db.refresh(novo_user)
    return {"status": "sucesso", "id": novo_user.id, "username": novo_user.username}

# Login de Utilizador - Verifica se o username existe e se a password está correta
@app.post("/api/auth/login")
def login_utilizador(dados: dict, db: Session = Depends(get_db)):
    username = dados.get("username")
    password = dados.get("password")

    user = db.query(Utilizador).filter(Utilizador.username == username).first()
    # Verifica se o utilizador existe e se a password introduzida é correta
    if not user or user.password_hash != encriptar_password(password):
        return {"status": "erro", "mensagem": "Credenciais inválidas."}

    return {"status": "sucesso", "id": user.id, "username": user.username}

# FUNÇÕES DA API MUSICAL
# Gera Exercício
@app.get("/api/exercicio/novo")
# Recebe os dados do filtro do browser (por defeito é Mistura)
def gerar_exercicio(filtro: str = "Mistura"):
    # Decide quais os números que podem ir a sorteio consoante o filtro
    opcoes_validas = []
    if filtro == "Intervalo":
        opcoes_validas = [0]
    elif filtro == "Escala":
        opcoes_validas = [1]
    elif filtro == "Tonalidade":
        opcoes_validas = [2]
    else:
        # Se for "Mistura", sorteia entre todos
        opcoes_validas = [0, 1, 2]
    # Sorteia o tipo de exercício a gerar
    sorteio = random.choice(opcoes_validas)

    if sorteio == 0:
        exercicio = gerar_intervalo_aleatorio()
        mensagem = "Qual é este intervalo?"
    elif sorteio == 1:
        exercicio = gerar_escala_aleatoria()
        # Verifica se o nome da resposta contém a palavra "Modo"
        if "Modo" in exercicio["detalhe"]:

```

```

        mensagem = "Qual o modo musical desta escala?"
    else:
        mensagem = "Qual é esta escala?"
    else:
        exercicio = gerar_exercicio_tonalidade()
        mensagem = exercicio["mensagem"]

# Constrói a resposta
resposta = {
    "status": "sucesso",
    "mensagem": mensagem,
    "tipo_exercicio": exercicio["tipo_exercicio"],
    "detalhe": exercicio["detalhe"],
    "notas": exercicio["notas"],
    "opcoes": exercicio["opcoes"],
    "explicacao": exercicio["explicacao"]
}

# Envia os dados da armação de clave se for um exercício de Tonalidade
if sorteio == 2:
    resposta["num_acidentes"] = exercicio["num_acidentes"]
    resposta["acidentes_ativos"] = exercicio["acidentes_ativos"]
return resposta

# Grava Resposta
@app.post("/api/tentativas")
def guardar_tentativa(tentativa: TentativaCreate, db: Session = Depends(get_db)):
    # Mapeia os dados recebidos, após validação, para o modelo do SQLAlchemy
    nova_tentativa = Tentativa(
        utilizador_id=tentativa.utilizador_id,
        tipo_exercicio=tentativa.tipo_exercicio,
        detalhe=tentativa.detalhe,
        resposta_dada=tentativa.resposta_dada,
        correta=tentativa.correta
    )

    # Guarda a tentativa no ficheiro SQLite
    db.add(nova_tentativa)
    db.commit()

    return {"status": "sucesso", "mensagem": "Gravado na Base de Dados!"}

# Dashboard de Métricas melhorado
@app.get("/api/dashboard/{user_id}")
def obter_metricas(user_id: int, db: Session = Depends(get_db)):
    # Ordenar por data - garante cálculo correto da evolução acumulada
    tentativas = db.query(Tentativa).filter(Tentativa.utilizador_id ==
user_id).order_by(Tentativa.data_hora.asc()).all()

    if not tentativas:
        return {"total": 0, "taxa_global": 0, "por_tipo": {}, "evolucao_diaria": []}

    total = len(tentativas)
    certas = sum(1 for t in tentativas if t.correta)

    # Taxa de acertos desagregada por Tipo
    tipos = {"Intervalo": [0,0], "Escala": [0,0], "Tonalidade": [0,0]}
    diario = {}

    for t in tentativas:
        if t.tipo_exercicio in tipos:
            tipos[t.tipo_exercicio][1] += 1
            if t.correta:
                tipos[t.tipo_exercicio][0] += 1

    # Agrupada por dia
    if hasattr(t, 'data_hora') and t.data_hora:
        data_str = t.data_hora.strftime("%Y-%m-%d")
    else:
        data_str = date.today().strftime("%Y-%m-%d") # Fallback

    if data_str not in diario:
        diario[data_str] = {"certas_dia": 0, "total_dia": 0}

    diario[data_str]["total_dia"] += 1
    if t.correta:
        diario[data_str]["certas_dia"] += 1

    por_tipo = {k: round((v[0]/v[1]*100), 1) if v[1] > 0 else 0 for k, v in tipos.items()}

    # Taxa por Dia e Acumulada
    evolucao_diaria = []
    certas_acumuladas = 0

```

```

total_acumulado = 0

for d in sorted(diario.keys()):
    c_dia = diario[d]["certas_dia"]
    t_dia = diario[d]["total_dia"]

    # Incrementa acumulado histórico
    certas_acumuladas += c_dia
    total_acumulado += t_dia

    taxa_dia = round((c_dia / t_dia * 100), 1) if t_dia > 0 else 0
    taxa_acum = round((certas_acumuladas / total_acumulado * 100), 1) if total_acumulado > 0 else 0

    evolucao_diaria.append({
        "data": d,
        "taxa_dia": taxa_dia,
        "taxa_acumulada": taxa_acum
    })

return {
    "total": total,
    "taxa_global": round((certas / total) * 100, 1),
    "por_tipo": por_tipo,
    "evolucao_diaria": evolucao_diaria
}

# Hard Reset - apaga todas as tentativas de um utilizador
@app.delete("/api/utilizadores/{user_id}/reset")
def reset_estatisticas(user_id: int, db: Session = Depends(get_db)):
    db.query(Tentativa).filter(Tentativa.utilizador_id == user_id).delete()
    db.commit()
    return {"status": "sucesso", "mensagem": "Estatísticas reiniciadas com sucesso."}

# SERVIDOR DE FICHEIROS
# Encontra a pasta onde ficheiro main.py está
caminho_atual = os.path.dirname(os.path.abspath(__file__))
# Constrói o caminho subindo um nível (..) e entrando na pasta frontend
caminho_frontend = os.path.join(caminho_atual, "..", "frontend")
# Diz ao FastAPI para servir o index.html a partir da pasta correta
app.mount("/", StaticFiles(directory=caminho_frontend, html=True), name="static")

```

BACKEND – MODELS.PY

```

# MODELOS DE DADOS - Define as tabelas e cria a base de dados SQLite.

# Importa biblioteca Python para trabalhar com bases de dados
from sqlalchemy import create_engine, Column, Integer, String, Boolean, DateTime, ForeignKey
from sqlalchemy.orm import declarative_base, relationship
# Importa biblioteca para lidar com datas e horas (registro temporal das tentativas)
from datetime import datetime, timezone
# Importa biblioteca para lidar com path de ficheiros
import os

# Encontra a pasta exata onde o models.py está
caminho_atual = os.path.dirname(os.path.abspath(__file__))
# Constrói o caminho absoluto para a base de dados dentro dessa pasta
caminho_bd = os.path.join(caminho_atual, "teoria_musical.db")

# Define o URL usando o caminho absoluto
SQLALCHEMY_DATABASE_URL = f"sqlite:/// {caminho_bd}"

# Motor que comunica com o SQLite.
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)

# Classe base genérica herdada por todas as tabelas
Base = declarative_base()

# Tabelas

class Utilizador(Base):

```



```

# Tabela que guarda a informação dos alunos/utilizadores.
__tablename__ = "utilizadores"

# Colunas da tabela
id = Column(Integer, primary_key=True, index=True) # Identificador único automático
username = Column(String, unique=True, index=True) # Nome de utilizador (não podem existir dois
iguais)
password_hash = Column(String) # Hash de segurança da password (evita que a password real seja
guardada)
data_registo = Column(DateTime, default=lambda: datetime.now(timezone.utc)) # Data e hora da criação
do perfil

# Relação: Um utilizador pode ter muitas 'tentativas' de exercícios
tentativas = relationship("Tentativa", back_populates="utilizador")

class Tentativa(Base):
    # Tabela que regista o histórico de exercícios resolvidos (Métricas do MVP).
    __tablename__ = "tentativas"

    # Chave primária e ligação ao utilizador (Chave Estrangeira)
    id = Column(Integer, primary_key=True, index=True)
    utilizador_id = Column(Integer, ForeignKey("utilizadores.id"))

    # Detalhes do exercício e desempenho
    tipo_exercicio = Column(String, index=True) # Ex: 'Escala', 'Intervalo', 'Tonalidade'
    detalhe = Column(String) # Ex: 'Dó Maior', '3ª Menor', 'Tonalidade'
    resposta_dada = Column(String) # Resposta do utilizador
    correta = Column(Boolean) # True se acertou, False se errou
    data_hora = Column(DateTime, default=lambda: datetime.now(timezone.utc)) # Registo temporal da
submissão

    # Relação: Cada tentativa pertence a um utilizador
    utilizador = relationship("Utilizador", back_populates="tentativas")

# Script de Inicialização
# Só corre se executarmos o ficheiro diretamente no terminal (python models.py).
# Lê as classes e cria o ficheiro físico 'teoria_musical.db' com as colunas certas.
if __name__ == "__main__":
    print("A criar a base de dados SQLite...")
    Base.metadata.create_all(bind=engine)
    print("Base de dados 'teoria_musical.db' criada com sucesso!")

```

BACKEND – MUSIC_LOGIC.PY

```

# Motor - Geração musical procedimental com Ortografia Diatónica e Distratores Inteligentes

import random

# Constantes para as Tonalidades - Ciclo das Quintas
ORDEM_SUSTENIDOS = ['F', 'C', 'G', 'D', 'A', 'E', 'B']
ORDEM_BEMOIS = ['B', 'E', 'A', 'D', 'G', 'C', 'F']

# Atribuição do número de acidentes para cada tonalidade
TONALIDADES_MAIORES = {
    -7: 'Cb Maior', -6: 'Gb Maior', -5: 'Db Maior', -4: 'Ab Maior', -3: 'Eb Maior', -2: 'Bb Maior', -1:
    'F Maior',
    0: 'C Maior',
    1: 'G Maior', 2: 'D Maior', 3: 'A Maior', 4: 'E Maior', 5: 'B Maior', 6: 'F# Maior', 7:
    'C# Maior'
}

TONALIDADES_MENORES = {
    -7: 'Ab Menor', -6: 'Eb Menor', -5: 'Bb Menor', -4: 'F Menor', -3: 'C Menor', -2: 'G Menor', -1: 'D
    Menor',
    0: 'A Menor',
    1: 'E Menor', 2: 'B Menor', 3: 'F# Menor', 4: 'C# Menor', 5: 'G# Menor', 6: 'D# Menor', 7:
    'A# Menor'
}

# O Dicionário de meios tons - Atribui a posição do som para a nota correta
DICIONARIO_ORTOGRAFICO = {
    0: {'c': {"vexflow": "c/4", "tone": "C4"}, 'b': {"vexflow": "b#/3", "tone": "B#3"}},

```

```

1: {'c': {"vexflow": "c#/4", "tone": "C#4"}, 'd': {"vexflow": "db/4", "tone": "Db4"}},
2: {'d': {"vexflow": "d/4", "tone": "D4"}},
3: {'d': {"vexflow": "d#/4", "tone": "D#4"}, 'e': {"vexflow": "eb/4", "tone": "Eb4"}},
4: {'e': {"vexflow": "e/4", "tone": "E4"}, 'f': {"vexflow": "fb/4", "tone": "Fb4"}},
5: {'f': {"vexflow": "f/4", "tone": "F4"}, 'e': {"vexflow": "e#/4", "tone": "E#4"}},
6: {'f': {"vexflow": "f#/4", "tone": "F#4"}, 'g': {"vexflow": "gb/4", "tone": "Gb4"}},
7: {'g': {"vexflow": "g/4", "tone": "G4"}},
8: {'g': {"vexflow": "g#/4", "tone": "G#4"}, 'a': {"vexflow": "ab/4", "tone": "Ab4"}},
9: {'a': {"vexflow": "a/4", "tone": "A4"}},
10: {'a': {"vexflow": "a#/4", "tone": "A#4"}, 'b': {"vexflow": "bb/4", "tone": "Bb4"}},
11: {'b': {"vexflow": "b/4", "tone": "B4"}, 'c': {"vexflow": "cb/5", "tone": "Cb5"}},
12: {'c': {"vexflow": "c/5", "tone": "C5"}, 'b': {"vexflow": "b#/4", "tone": "B#4"}},
13: {'c': {"vexflow": "c#/5", "tone": "C#5"}, 'd': {"vexflow": "db/5", "tone": "Db5"}},
14: {'d': {"vexflow": "d/5", "tone": "D5"}},
15: {'d': {"vexflow": "d#/5", "tone": "D#5"}, 'e': {"vexflow": "eb/5", "tone": "Eb5"}},
16: {'e': {"vexflow": "e/5", "tone": "E5"}, 'f': {"vexflow": "fb/5", "tone": "Fb5"}},
17: {'f': {"vexflow": "f/5", "tone": "F5"}, 'e': {"vexflow": "e#/5", "tone": "E#5"}},
18: {'f': {"vexflow": "f#/5", "tone": "F#5"}, 'g': {"vexflow": "gb/5", "tone": "Gb5"}},
19: {'g': {"vexflow": "g/5", "tone": "G5"}},
20: {'g': {"vexflow": "g#/5", "tone": "G#5"}, 'a': {"vexflow": "ab/5", "tone": "Ab5"}},
21: {'a': {"vexflow": "a/5", "tone": "A5"}}
}

# Alfabeto musical
SEQUENCIA_LETRAS = ['c', 'd', 'e', 'f', 'g', 'a', 'b']

# Notas limpas para gerar os exercícios
NOTAS_BASE_DISPONIVEIS = [
    {"som": 0, "letra": 0}, # Dó
    {"som": 2, "letra": 1}, # Ré
    {"som": 5, "letra": 3}, # Fá
    {"som": 7, "letra": 4}, # Sol
    {"som": 9, "letra": 5} # Lá
]

def formatar_nome_nota(nota_dict):
    # Auxiliar para remover a oitava (ex: C4 -> C, A#5 -> A#)
    return nota_dict["tone"].replace("3", "").replace("4", "").replace("5", "")

# Função para obter a nota correta a partir do dicionário
def obter_nota_ortografica(indice_absoluto, indice_letra):
    letra_desejada = SEQUENCIA_LETRAS[indice_letra % 7]
    opcoes_som = DICCIONARIO_ORTOGRAFICO.get(indice_absoluto, {})

    if letra_desejada in opcoes_som:
        return opcoes_som[letra_desejada]
    else:
        return list(opcoes_som.values())[0]

# Função para gerar um exercício de intervalo aleatório
def gerar_intervalo_aleatorio():
    base = random.choice(NOTAS_BASE_DISPONIVEIS)

    tipos_intervalos = {
        "2ª Menor": (1, 1), "2ª Maior": (2, 1),
        "3ª Menor": (3, 2), "3ª Maior": (4, 2),
        "4ª Perfeita": (5, 3), "4ª Aumentada": (6, 3),
        "5ª Perfeita": (7, 4),
        "6ª Menor": (8, 5), "6ª Maior": (9, 5),
        "7ª Menor": (10, 6), "7ª Maior": (11, 6),
        "Oitava": (12, 7)
    }

    # Distratores inteligentes para intervalos (Maior vs Menor, Perfeita vs Aumentada)
    distratores_diagnostics = {
        "2ª Menor": ["2ª Maior", "3ª Menor", "4ª Perfeita"],
        "2ª Maior": ["2ª Menor", "3ª Menor", "3ª Maior"],
        "3ª Menor": ["3ª Maior", "2ª Maior", "4ª Perfeita"],
        "3ª Maior": ["3ª Menor", "4ª Perfeita", "4ª Aumentada"],
        "4ª Perfeita": ["4ª Aumentada", "5ª Perfeita", "3ª Maior"],
        "4ª Aumentada": ["4ª Perfeita", "5ª Perfeita", "6ª Menor"],
        "5ª Perfeita": ["4ª Aumentada", "6ª Menor", "6ª Maior"],
        "6ª Menor": ["6ª Maior", "5ª Perfeita", "7ª Menor"],
        "6ª Maior": ["6ª Menor", "7ª Menor", "5ª Perfeita"],
        "7ª Menor": ["7ª Maior", "6ª Maior", "Oitava"],
        "7ª Maior": ["7ª Menor", "Oitava", "6ª Maior"],
        "Oitava": ["7ª Maior", "7ª Menor", "5ª Perfeita"]
    }

    nome_intervalo, regras = random.choice(list(tipos_intervalos.items()))
    meios_tons, saltos_letra = regras

    nota_base = obter_nota_ortografica(base["som"], base["letra"])

```

```

nota_alvo = obter_nota_ortografica(base["som"] + meios_tons, base["letra"] + saltos_letra)

# Injeção dos distratores
opcoes_resposta = distratores_diagnosticos[nome_intervalo] + [nome_intervalo]
random.shuffle(opcoes_resposta)

n_base = formatar_nome_nota(nota_base)
n_alvo = formatar_nome_nota(nota_alvo)
graus_distancia = saltos_letra + 1
explicacao = (
    f"A distância entre <strong>{n_base}</strong> e <strong>{n_alvo}</strong> é uma
<strong>{nome_intervalo}</strong>."
    f"Verifica que abrange exatamente <strong>{graus_distancia}</strong> graus</strong> na pauta (letras
consecutivas no alfabeto musical)"
    f"e uma separação acústica de <strong>{meios_tons}</strong> meio(s)-tom(ns)</strong>."
)

return {
    "tipo_exercicio": "Intervalo", "detalhe": nome_intervalo,
    "notas": [nota_base, nota_alvo], "opcoes": opcoes_resposta,
    "explicacao": explicacao
}

# Função para gerar um exercício de escala ou modo grego
def gerar_escalas_aleatoria():
    base = random.choice(NOTAS_BASE_DISPONIVEIS)

    tipos_escalas = {
        "Escala Maior (Jônio)": [0, 2, 4, 5, 7, 9, 11, 12],
        "Escala Menor Natural (Eólio)": [0, 2, 3, 5, 7, 8, 10, 12],
        "Escala Menor Harmônica": [0, 2, 3, 5, 7, 8, 11, 12],
        "Modo Dórico": [0, 2, 3, 5, 7, 9, 10, 12],
        "Modo Frígio": [0, 1, 3, 5, 7, 8, 10, 12],
        "Modo Lídio": [0, 2, 4, 6, 7, 9, 11, 12],
        "Modo Mixolídio": [0, 2, 4, 5, 7, 9, 10, 12],
        "Modo Lócrio": [0, 1, 3, 4, 6, 8, 10, 12]
    }

    # Distratores inteligentes para escalas (Agrupamento por familiaridade de modos)
    distratores_diagnosticos = {
        "Escala Maior (Jônio)": ["Modo Lídio", "Modo Mixolídio", "Escala Menor Natural (Eólio)",
        "Escala Menor Natural (Eólio)": ["Modo Dórico", "Escala Menor Harmônica", "Modo Frígio"],
        "Escala Menor Harmônica": ["Escala Menor Natural (Eólio)", "Modo Dórico", "Modo Frígio"],
        "Modo Dórico": ["Escala Menor Natural (Eólio)", "Modo Frígio", "Modo Mixolídio"],
        "Modo Frígio": ["Escala Menor Natural (Eólio)", "Modo Dórico", "Modo Lócrio"],
        "Modo Lídio": ["Escala Maior (Jônio)", "Modo Mixolídio", "Modo Frígio"],
        "Modo Mixolídio": ["Escala Maior (Jônio)", "Modo Dórico", "Modo Lídio"],
        "Modo Lócrio": ["Modo Frígio", "Escala Menor Natural (Eólio)", "Modo Dórico"]
    }

    nome_escalacorreta, padrao_meios_tons = random.choice(list(tipos_escalas.items()))

    notas_escalacorreta = []
    for salto_alfabeto, meios_tons in enumerate(padrao_meios_tons):
        nota = obter_nota_ortografica(base["som"] + meios_tons, base["letra"] + salto_alfabeto)
        notas_escalacorreta.append(nota)

    opcoes_resposta = distratores_diagnosticos[nome_escalacorreta] + [nome_escalacorreta]
    random.shuffle(opcoes_resposta)

    n = [formatar_nome_nota(nt) for nt in notas_escalacorreta]

    if nome_escalacorreta == "Escala Maior (Jônio)":
        explicacao = f"Na <strong>{nome_escalacorreta}</strong> de {n[0]}</strong>, os meios-tons estão
localizados estritamente entre o 3º/4º graus (<strong>{n[2]}-</strong>{n[3]}</strong>) e o 7º/8º graus
(<strong>{n[6]}-</strong>{n[7]}</strong>)."
    elif nome_escalacorreta == "Escala Menor Natural (Eólio)":
        explicacao = f"Na <strong>{nome_escalacorreta}</strong> de {n[0]}</strong>, os meios-tons estão
localizados estritamente entre o 2º/3º graus (<strong>{n[1]}-</strong>{n[2]}</strong>) e o 5º/6º graus
(<strong>{n[4]}-</strong>{n[5]}</strong>)."
    elif nome_escalacorreta == "Escala Menor Harmônica":
        explicacao = f"Na <strong>{nome_escalacorreta}</strong> de {n[0]}</strong>, encontra-se meios-tons entre o
2º/3º graus (<strong>{n[1]}-</strong>{n[2]}</strong>), 5º/6º graus (<strong>{n[4]}-</strong>{n[5]}</strong>) e 7º/8º graus
(<strong>{n[6]}-</strong>{n[7]}</strong>), além do característico intervalo de 2ª Aumentada (3 meios-tons) entre
o 6º e 7º graus (<strong>{n[5]}-</strong>{n[6]}</strong>)."
    elif nome_escalacorreta == "Modo Dórico":
        explicacao = f"Na <strong>{nome_escalacorreta}</strong> de {n[0]}</strong> (escala menor com 6ª Maior),
os meios-tons situam-se entre o 2º/3º graus (<strong>{n[1]}-</strong>{n[2]}</strong>) e o 6º/7º graus
(<strong>{n[5]}-</strong>{n[6]}</strong>)."
    elif nome_escalacorreta == "Modo Frígio":
        explicacao = f"Na <strong>{nome_escalacorreta}</strong> de {n[0]}</strong> (escala menor com a sua
marcante 2ª Menor), os meios-tons encontram-se imediatamente entre o 1º/2º graus (<strong>{n[0]}-</strong>
{n[1]}</strong>) e o 5º/6º graus (<strong>{n[4]}-</strong>{n[5]}</strong>)."

```

```

        elif nome_escala_correta == "Modo Lídio":
            explicacao = f"No <strong>{nome_escala_correta}</strong> de {n[0]}</strong> (escala maior com 4ª Aumentada), os meios-tons localizam-se entre o 4º/5º graus (<strong>{n[3]}-</strong>{n[4]}</strong>) e o 7º/8º graus (<strong>{n[6]}-</strong>{n[7]}</strong>)."
        elif nome_escala_correta == "Modo Mixolídio":
            explicacao = f"No <strong>{nome_escala_correta}</strong> de {n[0]}</strong> (escala maior com 7ª Menor dominante), os meios-tons encontram-se entre o 3º/4º graus (<strong>{n[2]}-</strong>{n[3]}</strong>) e o 6º/7º graus (<strong>{n[5]}-</strong>{n[6]}</strong>)."
        else: # Modo Lócrio
            explicacao = f"No <strong>{nome_escala_correta}</strong> de {n[0]}</strong> (modo diminuto com 5ª Diminuta), os meios-tons ocorrem entre o 1º/2º graus (<strong>{n[0]}-</strong>{n[1]}</strong>) e o 4º/5º graus (<strong>{n[3]}-</strong>{n[4]}</strong>)."

    return {
        "tipo_exercicio": "Escala", "detalhe": nome_escala_correta,
        "notas": notas_escala, "opcoes": opcoes_resposta,
        "explicacao": explicacao
    }

# Função para gerar um exercício de identificação de tonalidade por armação de clave
def gerar_exercicio_tonalidade():
    num_acidentes = random.randint(-7, 7)
    tipo_pergunta = random.choice(['Maior', 'Menor'])

    if tipo_pergunta == 'Maior':
        resposta_certa = TONALIDADES_MAIORES[num_acidentes]
    else:
        resposta_certa = TONALIDADES_MENORES[num_acidentes]

    acidentes_ativos = []
    if num_acidentes > 0:
        acidentes_ativos = [f"{nota}#" for nota in ORDEM_SUSTENIDOS[:num_acidentes]]
    elif num_acidentes < 0:
        acidentes_ativos = [f"{nota}b" for nota in ORDEM_BEMOIS[:abs(num_acidentes)]]

    # Distratores inteligentes para tonalidades (Erros comuns em Tonalidades)
    opcoes = [resposta_certa]

    # Distrator: Tonalidade Relativa (Mesma armação, modo oposto)
    if tipo_pergunta == 'Maior':
        opcoes.append(TONALIDADES_MENORES[num_acidentes])
    else:
        opcoes.append(TONALIDADES_MAIORES[num_acidentes])

    # Distrator: Erro de acidente (Ex: Confundir 2 sustenidos com 2 bemóis)
    if tipo_pergunta == 'Maior':
        opcoes.append(TONALIDADES_MAIORES[-num_acidentes])
    else:
        opcoes.append(TONALIDADES_MENORES[-num_acidentes])

    # Distrator: Erro de contagem de acidentes (Adicionar/subtrair 1 acidente)
    vizinho = num_acidentes + 1 if num_acidentes < 7 else num_acidentes - 1
    if num_acidentes == 0:
        vizinho = random.choice([-1, 1])

    if tipo_pergunta == 'Maior':
        opcoes.append(TONALIDADES_MAIORES[vizinho])
    else:
        opcoes.append(TONALIDADES_MENORES[vizinho])

    # Remove duplicados (ex: num_acidentes = 0 faz com que o oposto também seja 0)
    opcoes_unicas = list(set(opcoes))

    # Preenche com opções aleatórias mistas se faltarem devido a duplicados (Dó Maior / Lá Menor)
    while len(opcoes_unicas) < 4:
        aleatorio = random.choice(list(TONALIDADES_MAIORES.values()) + list(TONALIDADES_MENORES.values()))
        if aleatorio not in opcoes_unicas:
            opcoes_unicas.append(aleatorio)

    # Assegura que devolve apenas 4 e baralha
    opcoes_finais = opcoes_unicas[:4]
    random.shuffle(opcoes_finais)

    if num_acidentes == 0:
        texto_acidentes = "<strong>nenhum acidente</strong>"
    elif num_acidentes > 0:
        lista_acid = ", ".join(acidentes_ativos)
        texto_acidentes = f"<strong>{num_acidentes}</strong> sustenido(s)</strong> ({lista_acid})"
    else:
        lista_acid = ", ".join(acidentes_ativos)
        texto_acidentes = f"<strong>{abs(num_acidentes)}</strong> bemol/bemóis</strong> ({lista_acid})"

```

```

    explicacao = (
        f"A armação de clave apresentada contém exatamente {texto_acidentes}. "
        f"No Ciclo das Quintas, esta configuração exata define a assinatura de duas tonalidades relativas: "
        f"<strong>{TONALIDADES_MAIORES[num_acidentes]}</strong> (eixo Maior) ou <strong>{TONALIDADES_MENORES[num_acidentes]}</strong> (eixo Menor).")
    )

    return {
        "tipo_exercicio": "Tonalidade",
        "detalhe": resposta_certa,
        "notas": [],
        "opcoes": opcoes_finais,
        "num_acidentes": num_acidentes,
        "acidentes_ativos": acidentes_ativos,
        "mensagem": f"Qual é a Tonalidade {tipo_pergunta} com esta armação de clave?",
        "explicacao": explicacao
    }

```

BACKEND – TEST_MUSIC_LOGIC.PY

```

# Testes automatizados

import pytest
from music_logic import (
    obter_nota_ortografica,
    gerar_intervalo_aleatorio,
    gerar_escala_aleatoria,
    gerar_exercicio_tonalidade
)

# TESTES MUSICAIS E ORTOGRAFIA
def test_obter_nota_ortografica_diferenciador_enarmonico():
    # Garante que o motor distingue corretamente notas com o mesmo som mas escrita diferente.
    # Testa Dó natural (Som 0, Letra 0='c')
    nota_do = obter_nota_ortografica(0, 0)
    assert nota_do["tone"] == "C4"
    assert nota_do["vexflow"] == "c/4"

    # Testa Fá Sustenido (Som 6, Letra 3='f')
    nota_fa_sust = obter_nota_ortografica(6, 3)
    assert nota_fa_sust["tone"] == "F#4"
    assert nota_fa_sust["vexflow"] == "f#/4"

    # Testa Sol Bemol (Som 6, Letra 4='g') -> Mesmo som que Fá# mas com notação diferente
    nota_sol_bemol = obter_nota_ortografica(6, 4)
    assert nota_sol_bemol["tone"] == "Gb4"
    assert nota_sol_bemol["vexflow"] == "gb/4"

# TESTES DE GERAÇÃO DE INTERVALOS
def test_gerar_intervalo_aleatorio_estrutura():
    # Garante que a geração de intervalos devolve a estrutura e os tipos de dados corretos
    exercicio = gerar_intervalo_aleatorio()

    assert exercicio["tipo_exercicio"] == "Intervalo"
    assert len(exercicio["notas"]) == 2 # Um intervalo tem 2 notas
    assert len(exercicio["opcoes"]) == 4 # Têm de existir 4 opções de resposta (1 certa, 3 erradas)
    assert exercicio["detalhe"] in exercicio["opcoes"] # A resposta certa tem de estar nas opções
    assert isinstance(exercicio["explicacao"], str) # Tem de existir uma justificação teórica

# TESTES DE GERAÇÃO DE ESCALAS E MODOS
def test_gerar_escala_aleatoria_estrutura():
    # Garante que as escalas geradas têm 8 notas (1 oitava).
    exercicio = gerar_escala_aleatoria()

    assert exercicio["tipo_exercicio"] == "Escala"
    assert len(exercicio["notas"]) == 8 # Todas as escalas diatônicas implementadas têm 8 notas
    assert len(exercicio["opcoes"]) == 4
    assert exercicio["detalhe"] in exercicio["opcoes"]

    # Valida formato das notas
    primeira_nota = exercicio["notas"][0]

```

```

    assert "tone" in primeira_nota
    assert "vexflow" in primeira_nota

# TESTES DE GERAÇÃO DE TONALIDADES
def test_gerar_exercicio_tonalidade_estrutura():
    # Garante que as tonalidades geradas estão contidas nos limites do Ciclo das Quintas.
    exercicio = gerar_exercicio_tonalidade()

    assert exercicio["tipo_exercicio"] == "Tonalidade"
    assert -7 <= exercicio["num_acidentes"] <= 7 # 0 ciclo não excede 7 bemóis ou 7 sustenidos
    assert len(exercicio["opcoes"]) == 4
    assert exercicio["detalhe"] in exercicio["opcoes"]
    assert isinstance(exercicio["acidentes_ativos"], list)
    assert len(exercicio["notas"]) == 0 # Tonalidades testam a armação de clave, não têm notas de pauta

```